

Raw signal segmentation for estimating RNA modification from Nanopore direct RNA sequencing data

Reviewed Preprint

v2 • October 15, 2025

Revised by authors

Reviewed Preprint

v1 • March 6, 2025

Guangzhao Cheng, Aki Vehtari, Lu Cheng ✉

Department of Computer Science, Aalto University, Espoo, Finland • Institute of Biomedicine, University of Eastern Finland, Kuopio, Finland

https://en.wikipedia.org/wiki/Open_access

Copyright information

eLife Assessment

This study presents SegPore, a **valuable** new method for processing direct RNA nanopore sequencing data, which improves the segmentation of raw signals into individual bases and boosts the accuracy of modified base detection. The evidence presented to benchmark SegPore is **solid**, and the authors provide a fully documented implementation of the method. SegPore will be of particular interest to researchers studying RNA modifications.

<https://doi.org/10.7554/eLife.104618.2.sa4>

Abstract

Estimating RNA modifications from Nanopore direct RNA sequencing data is a critical task for the RNA research community. However, current computational methods often fail to deliver satisfactory results due to inaccurate segmentation of the raw signal. We have developed a new method, SegPore, which leverages a molecular jiggling translocation hypothesis to improve raw signal segmentation. SegPore is a pure white-box model with enhanced interpretability, significantly reducing structured noise in the raw signal. We demonstrate that SegPore outperforms state-of-the-art methods, such as Nanopolish and Tombo, in raw signal segmentation across three large benchmark datasets. Moreover, the improved signal segmentation achieved by SegPore enables SegPore+m6Anet to deliver state-of-the-art performance in site-level m6A identification. Additionally, SegPore surpasses baseline methods like CHEUI in single-molecule level m6A identification.

Introduction

RNA modifications play important roles in different diseases, such as Acute Myeloid Leukemia (1) and Fragile X Syndrome (2), as well as fundamental biological processes like cell differentiation (3,4) and immune response (5). To date, researchers have identified over 150 different types of RNA modifications (6–9), highlighting the complexity and diversity of

RNA regulation. These modifications are essential for proper RNA function, including maintaining secondary structures and facilitating accurate protein synthesis, such as the role of tRNA modifications in decoding mRNA codons (10 [↗](#)).

The practical applications of RNA modifications are far-reaching. For instance, N1-methylpseudouridine (m1Ψ) has been used to enhance the efficacy of COVID-19 mRNA vaccines, highlighting their therapeutic potential (11 [↗](#)). However, identifying and characterizing RNA modifications presents significant challenges due to the limitations of existing experimental techniques.

Traditional methods for RNA modification detection, such as MeRIP-Seq (12 [↗](#)), miCLIP (13 [↗](#)), and m6ACE-Seq (14 [↗](#)), rely on immunoprecipitation techniques that use modification-specific antibodies. While effective, these methods have several drawbacks. First, each method requires a separate antibody for each RNA modification, making it difficult to study multiple modifications simultaneously. Additionally, these techniques can only infer modification locations from next-generation sequencing (NGS) data, rather than measuring the modifications directly. As a result, they struggle to provide single-molecule resolution, limiting their accuracy and scope.

Recent advancements in direct RNA sequencing (DRS) by Oxford Nanopore Technologies (ONT) offer a promising alternative. DRS allows for the direct measurement of electrical currents as RNA molecules translocate through a nanopore, providing a potential avenue for detecting RNA modifications at the single-molecule level. Two versions of the direct RNA sequencing (DRS) kits are available: RNA002 and RNA004. Unless otherwise specified, this study focuses on RNA002 data. In this technique, five nucleotides (5mers) reside in the nanopore at a time, and each 5mer generates a characteristic current signal based on its unique sequence and chemical properties (16 [↗](#)). By analyzing these signals, it is possible to infer the original RNA sequence and detect modifications like N6-methyladenosine (m6A).

The general workflow of Nanopore direct RNA sequencing (DRS) data analysis is as follows. First, the raw electrical signal from a read is basecalled using tools such as Guppy or Dorado (<https://github.com/nanoporetech> [↗](#)), which produce the nucleotide sequence of the RNA molecule. However, these basecalled sequences do not include the precise start and end positions of each ribonucleotide (or k-mer) in the signal. Because basecalling errors are common, the sequences are typically mapped to a reference genome or transcriptome using minimap2 (15 [↗](#)) to recover the correct reference sequence. Next, tools such as Nanopolish (16 [↗](#), 17 [↗](#)) and Tombo (18 [↗](#)) align the raw signal to the reference sequence to determine which portion of the signal corresponds to each k-mer. We define this process as the segmentation task, referred to as “eventalign” in Nanopolish. Based on this alignment, Nanopolish extracts various features—such as the start and end positions, mean, and standard deviation of the signal segment corresponding to a k-mer. This signal segment or its derived features is referred to as an “event” in Nanopolish. The resulting events serve as input for downstream RNA modification detection tools such as m6Anet (19 [↗](#)) and CHEUI (20 [↗](#)). However, significant computational challenges remain. Segmenting the raw current signal into distinct 5mers and distinguishing between normal nucleotides and their modified counterparts is a complex task. Current methods, such as Nanopolish, employ models like Hidden Markov Models (HMM) to segment the signal, but they are prone to noise and inaccuracies, which degrade performance in downstream tasks like RNA modification prediction. The root of this issue lies in the fact that these methods do not accurately model the physical process of Nanopore sequencing, particularly the dynamics of the motor protein that drives RNA through the pore. As a result, the segmentation process is not well-aligned with the actual translocation mechanics, leading to signal noise that hinders precise modification detection.

This is where **SegPore**, our novel DRS segmentation and RNA modification prediction tool, steps in. SegPore was specifically developed to address the limitations of current methods by modeling the motor protein dynamics involved in RNA translocation. By incorporating these dynamics into

its segmentation algorithm, SegPore significantly reduces signal noise, enabling more accurate segmentation results. This cleaner segmentation foundation improves downstream analyses, allowing for more reliable RNA sequence and modification predictions. In SegPore, we first segment the raw signal into small fragments using a Hierarchical Hidden Markov Model (HHMM), where each fragment corresponds to a sub-state of a k-mer. Unlike Nanopolish and Tombo, which directly align the raw signal to the reference sequence, SegPore aligns the mean values of these small fragments to the reference. After alignment, we concatenate all fragments that map to the same k-mer into a larger segment, analogous to the “eventalign” output in Nanopolish. For RNA modification estimation, we use only the mean signal value of each reconstructed event.

Furthermore, a key limitation of current RNA modification tools, such as m6Anet, lies in the interpretability of their predictions. These deep neural network (DNN)-based methods classify modified and unmodified nucleotides using complex signal features that are not human-readable. This lack of interpretability can be problematic when applying these methods to new datasets, as researchers may struggle to trust the predictions without a clear understanding of how the results were generated. This severely limits their utility in real biological research, where transparency is critical.

In contrast, SegPore offers greater interpretability by using a simpler, more transparent Gaussian Mixture Model (GMM) to differentiate between modified and unmodified nucleotides. Instead of relying on complex, opaque features (see Supplementary Fig. S5), SegPore leverages baseline current levels to distinguish between, for example, m6A and normal adenosine. This straightforward approach allows researchers to easily understand the basis of modification predictions, making it more suitable for biological applications where confidence in the prediction process is essential.

By introducing SegPore, we bridge the gaps left by traditional tools. SegPore utilizes a hierarchical hidden Markov model (HHMM) for more precise segmentation and combines it with a GMM for RNA modification prediction, offering both greater accuracy and interpretability. This enables robust m6A modification detection at the single-molecule level, facilitating reliable, transparent predictions for both site-specific and molecule-wide m6A modifications.

Material and methods

SegPore workflow

Workflow overview

An overview of SegPore is illustrated in **Figure 1A** [↗](#), which outlines its five-step process. The output of Step 3 is the “events”, which is analogous to the output generated by the Nanopolish (v0.14.0) “eventalign” command and can be used as input for downstream models such as m6Anet. An “event” refers to a segment of the raw signal that is aligned to a specific k-mer on a read, along with its associated features such as start and end positions, mean current, standard deviation, and other relevant statistics. Step 4 allows for direct modification prediction at both the site and single-molecule levels. Notably, a key feature of SegPore is the 5mer parameter table, which defines the mean and standard deviation for each 5mer in either an unmodified or modified state. During the training phase, Steps 3~5 are iterated multiple times to stabilize the parameters in the 5mer table, which are subsequently fixed and applied for modification prediction on the test data. A detailed description of the model is provided in Supplementary Note 1.

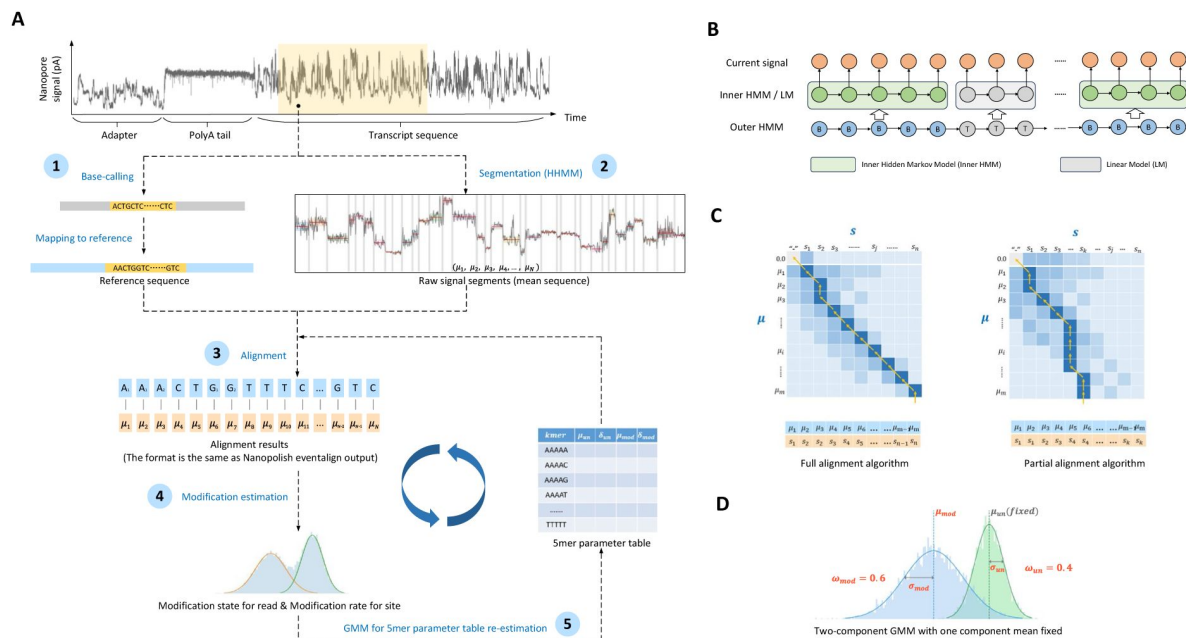


Figure 1.

SegPoRE workflow.

(A) General workflow. The workflow consists of five steps: (1) First, raw current signals are basecalled and mapped using Guppy and Minimap2. The raw current signal fragments are paired with the corresponding reference RNA sequence fragments using Nanopolish. (2) Next, the raw current signal of each read is segmented using a hierarchical hidden Markov model (HHMM), which provides an estimated mean (μ_i) for each segment. (3) These segments are then aligned with the 5mer list of the reference sequence fragment using a full/partial alignment algorithm, based on a 5mer parameter table. For example, A_j denotes the base "A" at the j -th position on the reference. (4) All signals aligned to the same 5mer across different genomic locations are pooled together, and a two-component Gaussian Mixture Model (GMM) is used to predict the modification at the site-level or single-molecule level. One component of the GMM represents the unmodified state, while the other represents the modified state. (5) GMM is used to re-estimate the 5mer parameter table. (B) Hierarchical hidden Markov model (HHMM). The outer HMM segments the current signal into alternating base and transition blocks. The inner HMM approximates the emission probability of a base block by considering neighboring 5mers. A linear model is used to approximate the emission probability of a transition block. (C) Full/partial alignment algorithms. Rows represent the estimated means of base blocks from the HHMM, and columns represent the 5mers of the reference sequence. Each 5mer can be aligned with multiple estimated means from the current signal. (D) Gaussian mixture model (GMM) for estimating modification states. The GMM consists of two components: the green component models the unmodified state of a 5mer, and the blue component models the modified state. Each component is described by three parameters: mean (μ), standard deviation (σ), and weight (ω).

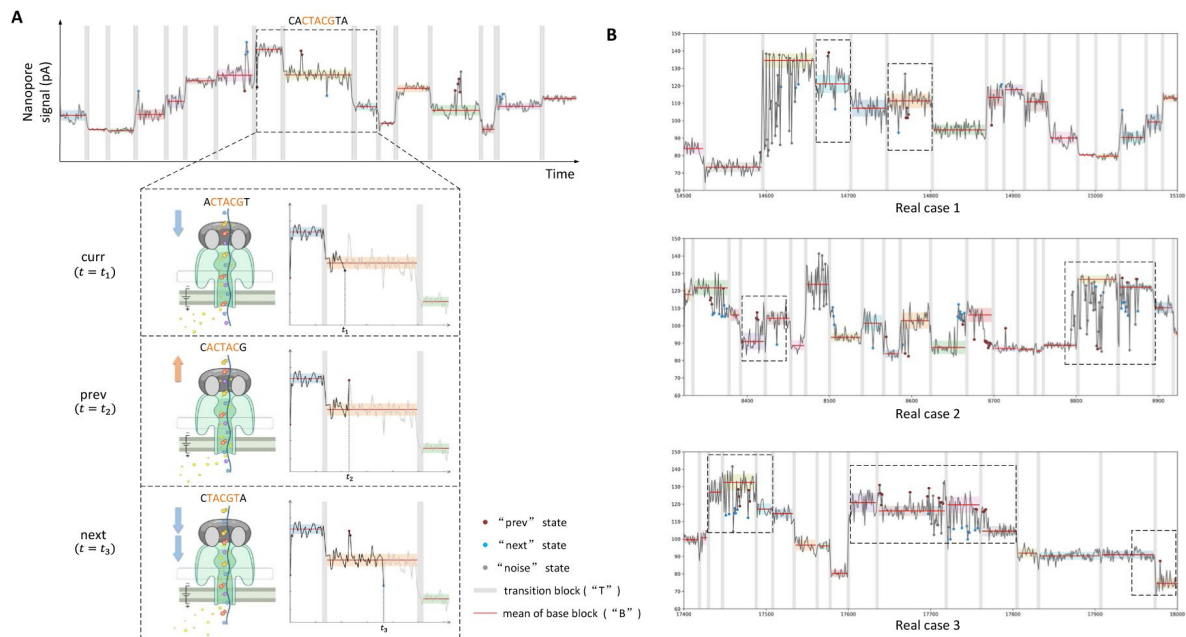


Figure 2.

RNA translocation hypothesis.

(A) Jiggling RNA translocation hypothesis. The top panel shows the raw current signal of Nanopore direct RNA sequencing, with gray areas representing SegPore-estimated transition blocks. We focus on three neighboring 5mers, considering the central 5mer (CTACG) as the current 5mer. The RNA molecule may briefly move forward or backward during the translocation of the current 5mer. If the RNA molecule is pulled backward, the previous 5mer is placed in the pore, and the current signal ("prev" state, red dots) resembles the previous 5mer's baseline (mean and standard deviation highlighted by red lines and shades). If the RNA is pushed forward, the current signal ("next" state, blue dots) is similar to the next 5mer's baseline. (B) Example raw current signals supporting the jiggling hypothesis. The dashed rectangles highlight base blocks, with red and blue points representing measurements corresponding to the previous and next 5mer, respectively. Red points align closely with the previous 5mer's baseline, and blue points match the next 5mer's baseline, reinforcing the hypothesis that the RNA molecule jiggles between neighboring 5mers. The raw current signals were extracted from mESC WT samples of the training data in the m6A benchmark experiment.

Preprocessing

We begin by performing basecalling on the input fast5 file using Guppy (v6.0.1), which converts the raw signal data into ribonucleotide sequences. Next, we align the basecalled sequences to the reference genome using Minimap2 (v2.28-r1209) ([15](#)), generating a mapping between the reads and the reference sequences. Nanopolish provides two independent commands: “polya” and “eventalign”. The “polya” command identifies the adapter, poly(A) tail, and transcript region in the raw signal, which we refer to as the poly(A) detection results. The raw signal segment corresponding to the poly(A) tail is used to standardize the raw signal for each read. The “eventalign” command aligns the raw signal to a reference sequence, assigning a signal segment to individual k-mers in the reference. It also computes summary statistics (e.g., mean, standard deviation) from the signal segment for each k-mer. Each k-mer together with its corresponding signal features is termed an event. These event features are then passed into downstream tools such as m6Anet and CHEUI for RNA modification detection. For full transcriptome analysis ([Fig. 3](#)), we extract the aligned raw signal segment and reference sequence segment from Nanopolish’s events for each read by using the first and last events as start and end points. For *in vitro* transcription (IVT) data with a known reference sequence ([Fig. 4](#)), we extract the raw signal segment corresponding to the transcript region for each input read based on Nanopolish’s poly(A) detection results.

Due to inherent variability between nanopores in the sequencing device, the baseline levels and standard deviations of k-mer signals can differ across reads, even for the same transcript. To standardize the signal for downstream analyses, we extract the raw current signal segments corresponding to the poly(A) tail of each read. Since the poly(A) tail provides a stable reference, we standardize the raw current signals across reads, ensuring that the mean and standard deviation of the poly(A) tail are consistent across all reads. This step is crucial for reducing variability between different reads and ensuring more accurate signal segmentation and modification prediction in subsequent steps. See Section 3 of Supplementary Note 1 for more details.

Signal segmentation via hierarchical Hidden Markov model

The RNA translocation hypothesis naturally leads to the use of a hierarchical Hidden Markov Model (HHMM) to segment the raw current signal. As shown in [Figure 1B](#), the HHMM consists of two layers. The outer HMM divides the raw signal into alternating base and transition blocks, represented by hidden states “B” (base) and “T” (transition). Within each base block, the inner HMM models the current signal at a more granular level.

The inner HMM includes four hidden states: “prev,” “next,” “curr,” and “noise.” These correspond to the previous, next, and current 5mer in the pore, while “noise” refers to random fluctuations. Each raw current measurement is emitted from one of these hidden states, providing a detailed model of the signal within the base blocks. A linear model with a large absolute slope is used to represent sharp changes in the transition blocks.

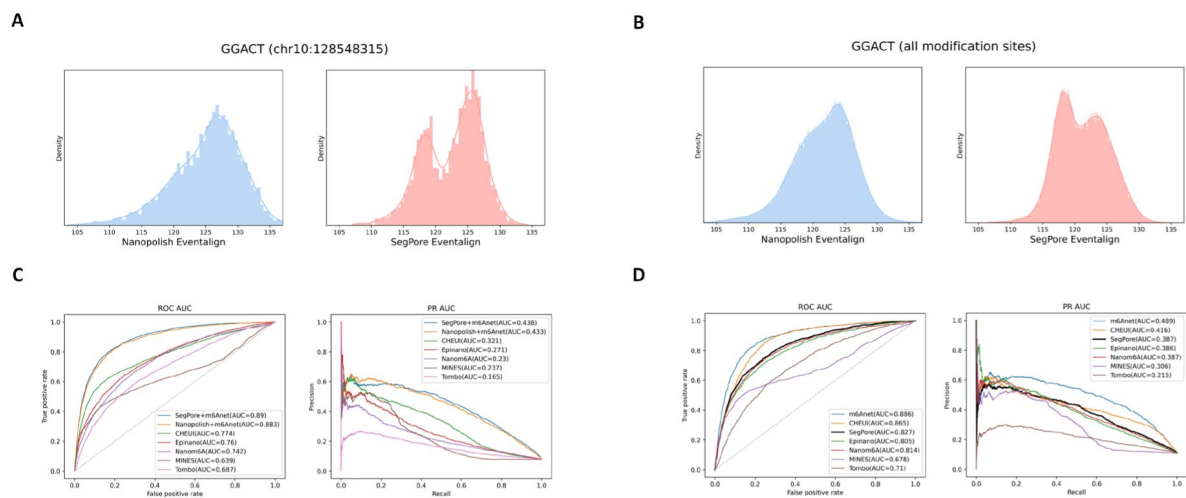


Figure 3.

m6A identification at the site level.

(A) Histogram of the estimated mean from current signals mapped to an example m6A-modified genomic location (chr10:128548315, GGACT) across all reads in the training data, comparing Nanopore (left) and SegPore (right). The x-axis represents current in picoamperes (pA). (B) Histogram of the estimated mean from current signals mapped to the GGACT motif at all annotated m6A-modified genomic locations in the training data, again comparing Nanopore (left) and SegPore (right). The x-axis represents current in picoamperes (pA). (C) Site-level benchmark results for m6A identification across all DRACH motifs, showing performance comparisons between SegPore+m6Anet and other methods. (D) Benchmark results for m6A identification on six selected motifs at the site level, comparing SegPore and other baseline methods.

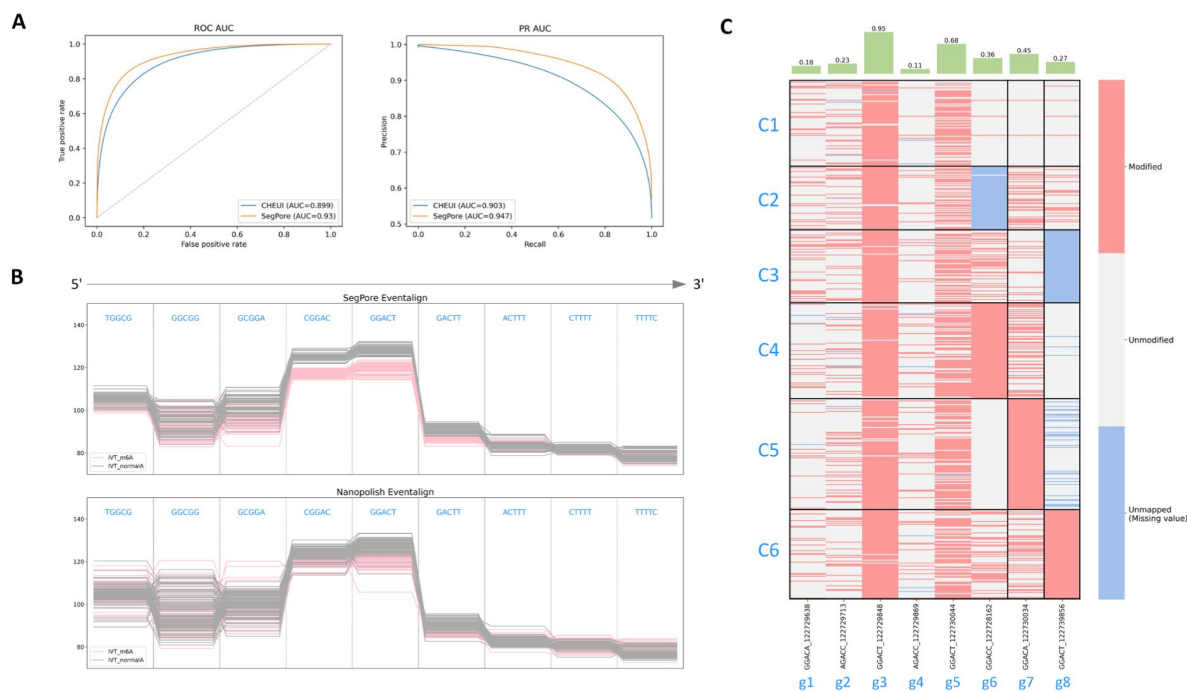


Figure 4.

m6A identification at the single-molecule level.

(A) Benchmark results for single-molecule m6A identification on IVT data. SegPore shows better performance compared to CHEUI in both PR-AUC and ROC-AUC. (B) Comparison of "eventalign" results from SegPore and Nanopolish for five consecutive k-mers. Note that DRS is sequenced from 3' to 5', so the k-mers enters the pore from right to left. A total of 100 reads were randomly sampled from transcript locations A1 (positions 711-719) in both the IVT_normalA and IVT_m6A samples (SRA: SRP166020). Each line represents an individual read, and the y-axis shows the raw signal intensity in picoampere (pA). Pink lines represent the IVT_m6A sample, and gray lines represent the IVT_normalA sample. The k-mers "GCGGA," "CGGAC," "GGACT," "GACTT," and "ACTTT" all contain N6-Methyladenosine (m6A) in the IVT_m6A sample. SegPore's results show clearer separation between m6A and normal adenosine, especially for "CGGAC" and "GGACT," compared to Nanopolish. (C) The upper panel shows the modification rate for selected genomic locations in the example gene ENSMUSG00000003153. The lower panel displays the modification states of all reads mapped to this gene. The black borders in the heatmap highlight the biclustering results, showing distinct modification patterns across different read clusters labeled C1 through C6.

To segment the signal, we first model the likelihood of the HHMM. Given the raw current signal y of a read, the hidden states of the outer hidden HMM are denoted by g . y and g are divided into $2K + 1$ blocks c , where $y^{(k)}$, $g^{(k)}$ correspond to k th block and $c = (c_1, c_2, \dots, c_k, \dots, c_{2K+1})$, $c_k \in \{B, T\}$. Blocks with odd indices ($k = 1, 3, 5, \dots, 2K + 1$) are base blocks, while those with even indices ($k = 2, 4, \dots, 2K$) are transition blocks. The likelihood of the HHMM is given by

$$p(y, g) = p(y|g)p(g) \quad (1)$$

$$= p(y|g)\{\pi_{g_1}^{outer} \prod_{i=2}^N T_{g_{i-1}g_i}^{outer}\} \quad (2)$$

$$= \{\prod_{i=1}^N p(y_i|g_i)\}\{\pi_{g_1}^{outer} \prod_{i=2}^N T_{g_{i-1}g_i}^{outer}\} \quad (3)$$

$$= \{\prod_{k=0}^K p(y^{(2k+1)}|c_{2k+1} = B)\}\{\prod_{k=1}^K p(y^{(2k)}|c_{2k} = T)\}\{\pi_{g_1}^{outer} \prod_{i=2}^N T_{g_{i-1}g_i}^{outer}\} \quad (4)$$

where $T_{g_{i-1}g_i}^{outer}$ is the transition matrix of the outer HMM and $\pi_{g_1}^{outer}$ is the probability for the first hidden state. It is obviously seen that the first part of the [Eq. 2](#) are emission probabilities, and the second part are the transition probabilities. It is not possible to directly compute the emission probabilities of the outer HMM ([Eq. 3](#)) since there exist dependencies for the current signal measurements within a base or transition block. Therefore, we use the inner HMM and linear model ([Eq. 4](#)) to handle the dependencies and approximate emission probabilities.

The inner HMM models transitions between the “prev,” “next,” “curr,” and “noise” states. For the “prev,” “next,” and “curr” states, a Gaussian distribution is used to model the emission probabilities, while a uniform distribution models the “noise” state. The forward-backward algorithm is used to compute the marginal likelihood of the inner HMM, which approximates the emission probabilities for base blocks. For transition blocks, a standard linear model computes the emission probabilities.

For any given g , we can calculate the joint likelihood ([Eq. 1](#)). We enumerate different configurations and select the one with the highest likelihood. This process segments the raw current signal into alternating base and transition blocks, where one or more base blocks may correspond to a single 5mer. Each base block is characterized by its mean and standard deviation, which are used as input for downstream alignment tasks.

Due to the large size of fast5 files (which can reach terabytes), parameter inference for this model is computationally intensive. To address this, we developed a GPU-based inference algorithm that significantly accelerates the process. More details on this algorithm can be found in Supplementary Note 2.

In summary, the HHMM allows us to accurately segment the raw current signal, providing estimates of the mean and variance for each base block, which are crucial for downstream analyses such as RNA modification prediction.

5mer parameter table

We downloaded the k-mer models “r9.4_180mv_70bps_5mer_RNA” from the ONT GitHub repository (https://github.com/nanoporetech/kmer_models). The columns labeled “level_mean” and “level_stdv” in these models were used as the mean and standard deviation values for the unmodified 5mers. These values serve as the initial parameters in the 5mer parameter table for SegPore, which we refer to as the *ONT 5mer parameter table*.

The initialization of the 5mer parameter table is a critical step in SegPore’s workflow. By leveraging ONT’s established k-mer models, we ensure that the initial estimates for unmodified 5mers are grounded in empirical data. These initial estimates are refined through SegPore’s

iterative parameter estimation process, enabling the model to accurately differentiate between modified and unmodified 5mers during segmentation and modification prediction tasks. The refined 5mer parameters also provide a foundation for downstream analysis, such as alignment and m6A identification.

Alignment of raw signal segment with reference sequence

After segmenting the raw current signal of a read into base and transition blocks using HHMM, we align the means of base blocks with the 5mer list of the reference sequence.

The alignment process involves three main cases:

1. Base block matching with a 5mer: In this case, a base block aligns with a 5mer, which is modeled by a Gaussian distribution. The 5mer may exist in either an unmodified or modified state, and the corresponding Gaussian parameters are retrieved from the 5mer parameter table.
2. Base block matching with an insertion: Here, the base block aligns with an indel (“-”), indicating an inserted nucleotide in the read.
3. Deletion in the read: In this case, an indel (0.0) matches with a 5mer, representing a deleted nucleotide in the read.

The alignment score function models these matching cases as follows. For the first case (a base block matching a 5mer), we calculate the probability that the base block mean is sampled from either the unmodified or modified 5mer’s Gaussian distribution. The larger of the two probabilities is used as the match score. For the second and third cases, the alignment is treated as noise, and a fixed uniform distribution is used to calculate the match score.

An important distinction from classical global alignment algorithms is that one or multiple base blocks may align with a single 5mer. Given the base block means $\mu = (\mu_1, \mu_2, \dots, \mu_p, \dots, \mu_m)$ and 5mer list $s = (s_1, s_2, \dots, s_p, \dots, s_n)$, we define $(m + 1) \times (n + 1)$ the score matrix as M (Fig. 1C). The first row and column of the matrix are reserved for indels (“0.0” and “-”), representing insertions or deletions in the base blocks or 5mers, respectively.

We denote the score function by f . The recursion formula of the dynamic programming alignment algorithm is given by

$$M(i, j) = \max \begin{cases} M(i-1, j-1) + f(\mu_i, s_j) \\ M(i, j-1) + f(0.0, s_j) \\ M(i-1, j) + f(\mu_i, -) \\ M(i-1, j) + f(\mu_i, s_j) \end{cases} \quad (5)$$

, where f is the score function. It can be seen that we can still align μ_i with s_j after we have aligned μ_{i-1} with s_j , which fulfills the special consideration that one or multiple base blocks might be aligned with one 5mer.

We implement two types of alignment algorithms (Fig. 1C) based on the score matrix M :

1. Full Alignment Algorithm: This algorithm aligns the full list of base block means with the full list of 5mers, similar to classical global alignment. It traces back from the $(m + 1, n + 1)$ position in the score matrix.
2. Partial Alignment Algorithm: This aligns the full list of base blocks with the initial part of the 5mer list, with no indels allowed in the base block means or the 5mer list. The trace back starts from the maximum value in the last row of the score matrix.

A detailed description of both alignment algorithms is provided in Supplementary Note 1. The output of the alignment algorithm is an alignment that pairs the base blocks with the 5mers from the reference sequence for each read (**Fig. 1C**). Base blocks aligned to the same 5mer are concatenated into a single raw signal segment (referred to as an “event”), from which various features—such as start and end positions, mean current, and standard deviation—are extracted. Detailed derivation of the mean and standard deviation is provided in Section 5.3 in Supplementary Note 1. In the remainder of this paper, we refer to these resulting events as the output of eventalign analysis or the segmentation task.

Modification prediction

After obtaining the eventalign results, we estimate the modification state of each motif using the 5mer parameter table. Specifically, for each 5mer, we compare the probability that its mean is sampled from either the modified or unmodified 5mer’s Gaussian distribution. If the probability under the modified 5mer distribution is higher, the 5mer is predicted to be in the modification state, and vice versa for the unmodified state.

To estimate the overall modification rate at a specific genomic location, we pool all reads that map to that location on the reference sequence. The modification rate is calculated as the proportion of reads that are predicted to be in the modification state at that location. A detailed description of the modification state prediction process can be found in Supplementary Note 1.

GMM for 5mer parameter table re-estimation

To improve alignment accuracy and enhance modification predictions, we use a Gaussian Mixture Model (GMM) to iteratively fine-tune the 5mer parameter table. As illustrated in **Figure 1A**, the rows of the 5mer parameter table represent the 5mers, while the columns provide the mean and standard deviation for both the unmodified and modified states. We denote a 5mer by s , with its relevant parameters as $\mu_{s,on}$, $\delta_{s,on}$, $\mu_{s,mom}$, $\delta_{s,mom}$.

Using the alignment results from all reads, we collect the base block means aligned to the same 5mer (denoted by ss) across different reads and genomic locations, particularly those with high modification rates. A two-component GMM is then fit to these base blocks. In this process, the mean of the first component is fixed to $\mu_{s,oonn}$. From the GMM, we update the parameters $\delta_{s,on}$, $\omega_{s,on}$, $\mu_{s,mom}$, $\delta_{s,mom}$, and $\omega_{s,mom}$, where $\omega_{s,on}$, $\omega_{s,mom}$ represent the weights for the unmodified and modified components, respectively. Afterward, we manually adjust the 5mer parameter table using heuristics to ensure that the modified 5mer distribution is significantly distinct from the unmodified distribution (see details in Section 7 of Supplementary Note 1).

This re-estimation process is only performed on the training data. The initial 5mer parameter table is based on the table provided by ONT. After each iteration of updating the table, we rerun the SegPore workflow from the alignment step onward. Typically, the process is repeated three to five times until the 5mer parameter table stabilizes. Once a stabilized 5mer parameter table is estimated from the training data, it is used for RNA modification estimation in the test data without further updates. A more detailed description of the GMM re-estimation process is provided in Section 6 of Supplementary Note 1.

m6A site level benchmark

The HEK293T wild-type (WT) samples were downloaded from the ENA database (accession number PRJEB40872), while the HCT116 samples were obtained from ENA under accession PRJEB44348. The reference sequence (Homo_sapiens.GRCh38.cdna.ncrna_wtChrIs_modified.fa) was downloaded from <https://doi.org/10.5281/zenodo.4587661>. Ground truth data were sourced

from Supplementary Data 1 of Pratanwanich, P.N. et al. (21 [DOI](#)). Fast5 files for the test dataset (mESC WT samples, mESCs_Mettl3_WT_fast5.tar.gz) (22 [DOI](#)) were retrieved from the NCBI Sequence Read Archive (SRA) under accession SRP166020.

During training, we initialized the 5mer parameter table using ONT's data. The standard SegPore workflow was executed on the training data (HEK293T WT samples), using the full alignment algorithm. The 5mer parameter table estimation was iterated five times. For mapping, reads were first aligned to the cDNA and subsequently converted to genomic locations using Ensembl GTF file (GRCh38, v9). The same 5mer across different genomic locations was pooled together. A 5mer was considered significantly modified if its read coverage exceeded 1,500 and the distance between the means of the two Gaussian components in the GMM was greater than 5 picoamperes (pA). As a result, modification parameters were specified for ten significant 5mers, as illustrated in Supplementary Fig. S2A.

With the estimated 5mer parameter table from the training data, we then ran the SegPore workflow on the test data. The transcript sequences from GENCODE release version M18 were used as the reference sequence for mapping, with the corresponding GTF file (gencode.vM18.chr_patch_hapl_scaff.annotation.gtf) downloaded from GENCODE to convert transcript locations into genomic coordinates. It is important to note that the 5mer parameter table was not re-estimated for the test data. Instead, modification states for each read were directly estimated using the fixed 5mer parameter table. Due to the differences between human (Supplementary Fig. S2A) and mouse (Supplementary Fig. S2B), only six 5mers were found to have m6A annotations in the test data's ground truth (Supplementary Fig. S2C). For a genomic location to be identified as a true m6A modification site, it had to correspond to one of these six common 5mers and have a read coverage of greater than 20. SegPore derived the ROC and PR curves for benchmarking based on the modification rate at each genomic location.

In the SegPore+m6Anet analysis, we fine-tuned the m6Anet model using SegPore's eventalign results to demonstrate improved m6A identification. We started with the pre-trained m6Anet model (available at <https://github.com/GoekeLab/m6anet> [DOI](#), model version: HCT116_RNA002) and fine-tuned it using the eventalign results from HCT116 samples. SegPore's eventalign output provided the pairing between each genomic location and its corresponding raw signal segment, which allowed us to extract normalized features such as the normalized mean $\mu_{i,p}$, standard deviation $\sigma_{i,p}$, dwell time $l_{i,p}$. For genomic location i , m6Anet extracts a feature vector $m_i = \{\mu_{i-1}, \sigma_{i-1}, l_{i-1}, \mu_p, \sigma_p, l_p, \mu_{i+1}, \sigma_{i+1}, l_{i+1}\}$, which was used as input for m6Anet. Feature vectors from 80% of the genomic locations were used for training, while the remaining 20% were set aside for validation. We ran 100 epochs during fine-tuning, selecting the model that performed best on the validation set.

Ground truth data and the performance of other methods (Tombo v1.5.1, Nanom6A v2.0, m6Anet v1.0, and Epinao v1.2.0) on the mESC dataset were provided through personal communications with Prof. Luo Guanzheng, the corresponding author of the benchmark study referenced (23 [DOI](#)).

m6A single molecule level benchmark

The benchmark IVT data for single molecule m6A identification was downloaded from NCBI-SRA with accession number SRP166020. CHEUI was used for the benchmark. For the ground truth, every A in every read of the IVT_m6A sample is treated as a m6A modification and every A in every read of the IVT_normalA is treated as a normal A. Detailed methods for calculating modification probability at single molecule level were provided in Supplementary Note 1, Section 6.1-6.2.

Results

RNA translocation hypothesis

Accurate segmentation of raw current signals in direct RNA sequencing remains a major challenge, largely because the precise dynamics of RNA translocation through the pore are not fully understood. To address this, we propose a hypothesis that better reflects the physical movement of the RNA molecule. In traditional basecalling algorithms such as Guppy and Albacore, we implicitly assume that the RNA molecule is translocated through the pore by the motor protein in a monotonic fashion, i.e., the RNA is pulled through the pore unidirectionally. In the DNN training process of Guppy and Albacore, we try to align the current signal with the reference RNA sequence. The alignment is unidirectional, which is the source of the implicit monotonic translocating assumption.

Contrary to the conventional assumption of monotonic translocation, the raw current data suggests that the motor protein drives RNA both forward and backward during sequencing. **Figure 2B** illustrates this with several example fragments of DRS raw current signal (21), where each fragment roughly corresponds to three neighboring 5mers. The raw current signals, as shown in **Figure 2B**, strongly support this hypothesis, with several instances of measured current intensities matching both the previous and next 5mer's baseline. These repeated patterns, observed across multiple DRS datasets, provide empirical evidence of the jiggling translocation. Similar patterns are widely observed across the whole data. This suggests that the RNA molecule may move forward and backward while passing through the pore. This observation is also supported by previous reports (24,25), in which the helicase (the motor protein) translocates the DNA strand through the nanopore in a back-and-forth manner. Depending on ATP or ADP binding, the motor protein may translocate the DNA/RNA forward or backward by 0.5~1 nucleotides.

Based on the reported kinetic model (24), we hypothesize that the RNA is translocated through the pore in a jiggling manner. This “jiggling” hypothesis presents a significant departure from the traditionally accepted view of unidirectional RNA translocation and aligns better with recent evidence from studies on DNA translocation (24,25). Incorporating this hypothesis into segmentation algorithms could lead to more accurate predictions of RNA modifications by accounting for the dynamic nature of translocation. On average, the motor protein sequentially translocates 5mers on the RNA strand forward, and each 5mer resides in the pore for a short time. During the short period of a single 5mer, the motor protein may swiftly drive the RNA molecule forward and backward by 0.5~1 nucleotide in the translocation process of the current 5mer (**Fig. 2A**), which makes the measured current intensity occasionally similar to the previous or the next 5mer. When the motor protein does not move the RNA molecule, we hypothesize that the RNA molecule undergoes slight thermal fluctuations, causing it to oscillate slightly within the pore and produce a stable current close to its baseline. In contrast, sharp changes in current intensity between consecutive 5mers define transition blocks, where one 5mer is replaced by the next.

We also assume that the raw current signal of a read can be segmented into a series of alternating base and transition blocks. In the ideal case, a base block corresponds to the base state where the 5mer resides in the pore and jiggles between neighboring 5mers, i.e., the current 5mer can transiently jump to the previous or the next 5mer. A transition block corresponds to the transition state between two consecutive base states where one 5mer translocates to the next 5mer in the pore. The current signal should be relatively flat in the base blocks, while a sharp change is expected in the transition blocks. One challenge we encountered was the overestimation of transition blocks. This may be due to a base block actually corresponding to a sub-state of a single 5mer, rather than each base block corresponding to a full 5mer, leading to inflated transition

counts. To address this issue, SegPore's alignment algorithm was refined to merge multiple base blocks (which may represent sub-states of the same 5mer) into a single 5mer, thereby facilitating further analysis.

SegPore Workflow

The SegPore workflow (**Fig. 1**) consists of five key steps: (1) Preprocess fast5 files to pair raw current signal segments with corresponding RNA sequence fragments; (2) Segment each raw current signal using a hierarchical hidden Markov model (HHMM) into base and transition blocks; (3) Align the derived base blocks with the paired RNA sequence; (4) Fit a two-component GMM to estimate the modification state at the single-molecule level or the modification rate at the site level; (5) Use the results from Step (4) to update relevant parameters. Steps (3) to (5) are iterative and continue until the estimated parameters stabilize.

The final outputs of SegPore are the events and modification state predictions. SegPore's events are similar to the outputs of Nanopolish's "eventalign" command, in that they pair raw current signal segments with the corresponding RNA reference 5mers. Each 5mer is associated with various features — such as start and end positions, mean current, and standard deviation — derived from the paired signal segment. For selected 5mers that exhibit one clearly unmodified component and one clearly modified component, SegPore reports the modification rate at each site, as well as the modification state of that site on individual reads.

A key component of SegPore is the 5mer parameter table, which specifies the mean and standard deviation for each 5mer in both modified and unmodified states (**Fig. 1A**). Since the peaks (representing modified and unmodified states) are separable for only a subset of 5mers, SegPore can provide modification parameters for these specific 5mers. For other 5mers, modification state predictions are unavailable.

Segmentation benchmark

To evaluate SegPore's performance in raw signal segmentation, we compared SegPore with Nanopolish (v0.14.0) and Tombo (v1.5.1) using three Nanopore direct RNA sequencing (DRS) datasets: two HEK293T datasets (wild type and Mettl3 knockout) (21) and the HCT116 dataset (26). Nanopolish and SegPore employed the "eventalign" method to align 5mers on each read with their corresponding raw signals, producing the mean and standard deviation (std) of the aligned signal segments. Tombo used the "resquiggle" method to segment the raw signals, and we standardized the segments using the poly(A) tail to ensure a fair comparison (See preprocessing section in Materials and Methods).

To benchmark segmentation performance, we used two key metrics (details provided in Supplementary Note 1, Section 8): (1) the log-likelihood of the segment mean, which measures how closely the segment matches ONT's 5mer parameter table (used as ground truth), and (2) the standard deviation (std) of the segment, where a lower std indicates reduced noise and better segmentation quality. If the raw signal segment aligns correctly with the corresponding 5mer, its mean should closely match ONT's reference, yielding a high log-likelihood. A lower std of the segment reflects less noise and better performance overall.

As shown in **Table 1**, SegPore consistently achieved the best performance averaged on all 5mers across all datasets, with the highest log-likelihood and the lowest std values. These results suggest that SegPore provides a more accurate segmentation of the raw signal with significantly reduced noise compared to Nanopolish and Tombo.

To provide a more intuitive comparison, the segmentation results for two example raw signal clips are illustrated in Supplementary Fig. S1. These examples demonstrate the clearer, more precise segmentation achieved by SegPore compared to Nanopolish.

Test Dataset	Avg. std ($\hat{\sigma}$) ↓			Avg. log p ($\hat{L}$) ↑		
	Nanopolish	Tombo	SegPore	Nanopolish	Tombo	SegPore
HEK293T(WT)	3.073	4.187	2.736	-2.871	-3.749	-2.778
HEK293T(KO)	2.948	4.204	2.670	-2.856	-3.749	-2.745
HCT116	3.167	4.076	2.872	-2.872	-3.704	-2.746

Table 1.
Segmentation benchmark on RNA002 data

To evaluate SegPore's performance on RNA004 data, we compared it with f5c (v1.5) (27) and Uncalled4 (28) (v4.1.0) using three public DRS datasets: the *S. cerevisiae* dataset (29), the curlcake IVT and m6A datasets (30). RNA004 data differs from RNA002 in two key aspects: RNA002 provides reference current levels for 5mers, typically ranging from 60 to 140 pA, whereas RNA004 provides reference values for 9mers, normalized to approximately zero mean and unit variance. As there are currently no established poly(A) detection methods available for RNA004, we used f5c to standardize the raw signals of each read prior to segmentation. SegPore was then applied to perform segmentation on the standardized signals. We computed the same benchmarking metrics—average log-likelihood and standard deviation—using the standardized raw signals and the segmentation results from f5c, Uncalled4, and SegPore. As shown in **Table 2**, SegPore achieved the best overall performance across all datasets, indicating its robustness and suitability for RNA004 data. Moreover, we find that the jiggling hypothesis remains valid for RNA004, as illustrated in Supplementary Fig. S4.

m6A identification at the site level

We evaluated SegPore's performance in raw signal segmentation and m6A identification using independent public datasets as both training and test data. Since m6A modifications typically occur at DRACH motifs (where D denotes A, G, or U, and H denotes A, C, or U) (13), this study focuses on estimating m6A modifications on these motifs.

To begin, we estimated the 5mer parameter table for m6A modifications using Nanopore direct RNA sequencing (DRS) data from three wild-type human HEK293T cell samples (21). The fast5 files from all samples were concatenated, and the full SegPore workflow was run to obtain the 5mer parameter table. The parameter estimation process was iterated five times to ensure stabilization, refining the modification parameters for ten 5mers where the modification state distribution significantly differs from the unmodified state.

Next, we applied SegPore's segmentation and m6A identification to test data from wild-type mouse embryonic stem cells (mESCs) (23). Given the comparable methods and input data requirements, we benchmarked SegPore against several baseline tools, including Tombo, MINES (31), Nanom6A (32), m6Anet, Epinao (33), and CHEUI (20). Based on the output of SegPore eventalign, we fine-tune m6Anet using the HCT116 data at all DRACH motifs, aiming to demonstrate that the performance of SegPore benefits downstream models. Additionally, due to the differences of the k-mer motifs between human and mouse (Supplementary Fig. S2), six shared 5mers were selected to demonstrate SegPore's performance in modification prediction directly. By utilizing the 5mer parameter table derived from the training data, SegPore employs a two-component GMM to calculate the modification rates at the selected m6A sites.

SegPore demonstrated improved segmentation compared to Nanopolish. **Figure 3A** shows the eventalign results at an example genomic location with m6A modifications. SegPore's results show a more pronounced bimodal distribution in the raw signal segment mean, indicating clearer separation of modified and unmodified signals. Furthermore, when pooling all reads mapped to m6A-modified locations at the GGACT motif, SegPore exhibited more distinct peaks (**Fig. 3B**), indicating reduced noise and potentially enabling more reliable modification detection.

We evaluated m6A predictions using two approaches: (1) SegPore's segmentation results were fed into m6Anet, referred to as SegPore+m6Anet, which works for all DRACH motifs and (2) direct m6A predictions from SegPore's Gaussian Mixture Model (GMM), which is limited to the six selected 5mers that exhibit clearly separable modified and unmodified components in the GMM (see Materials and Methods for details).

Test Dataset	Avg. std ($\hat{\sigma}$) ↓			Avg. log p ($\hat{L}$) ↑		
	f5c	Uncalled4	SegPore	f5c	Uncalled4	SegPore
<i>S. cerevisiae</i>	3.129	3.970	3.125	-2.541	-3.835	-2.489
curlcake(IVT)	3.424	4.729	3.359	-2.716	-2.904	-2.515
curlcake(m6A)	3.520	4.971	3.392	-3.118	-3.524	-2.599

Table 2.

Segmentation benchmark on RNA004 data

In terms of m6A identification, SegPore performed strongly on the test data. Using miCLIP2 (34) data as the ground truth, we calculated the area under the curve (AUC) for both the receiver operating characteristic (ROC) and precision-recall (PR) curves. SegPore+m6Anet achieved the best performance with an ROC AUC of 89.0% and a PR AUC of 43.8% (Fig. 3C). For six selected m6A motifs, SegPore achieved an ROC AUC of 82.7% and a PR AUC of 38.7%, earning the third best performance compared with deep learning methods m6Anet and CHEUI (Fig. 3D). It is noteworthy that SegPore's GMM for m6A estimation is a very simple model, utilizing far fewer parameters than DNN-based methods. Achieving the decent performance with such a simple model is a significant accomplishment. These results highlight SegPore's robust performance in m6A identification.

m6A identification at the single molecule level

SegPore naturally identifies m6A modifications at the single-molecule level, which is crucial for understanding the heterogeneity of RNA modifications across individual transcripts. We benchmarked SegPore against CHEUI using an *in vitro* transcription (IVT) dataset containing two samples—one transcribed with m6A and the other with normal adenosine (22). This dataset provides clear ground truth for m6A modifications at the single-molecule level, with all adenosine positions replaced by m6A in the *ivt_m6A* sample, and all adenosines unmodified in the *ivt_normalA* sample. We used 60% of the data for training and 40% for testing, with both SegPore and CHEUI estimating the m6A modification probability at each adenosine site on each read. Based on these probabilities, we calculated the ROC-AUC and PR-AUC by varying the modification probability threshold.

As shown in Figure 4A, SegPore outperformed CHEUI on this benchmark dataset, achieving better performance in both PR-AUC (94.7% vs 90.3%) and ROC-AUC (93% vs 89.9%). These results clearly demonstrate SegPore's accuracy and robustness in detecting single-molecule m6A modifications.

Next, we demonstrate SegPore's interpretability through an example comparison of raw signal clips from both *ivt_m6A* and *ivt_normalA* samples (Fig. 4B). The raw signal segments (means) are aligned to a randomly selected m6A site as well as its neighbouring sites in the reference sequence. Each line represents an individual read, with pink lines from the *ivt_m6A* sample and gray lines from the *ivt_normalA* sample. SegPore's segmentation clearly distinguishes between m6A and normal adenosine at the single-molecule level, while Nanopolish's segmentation shows less distinction.

Interestingly, we observed that the position of m6A within a 5mer can affect the signal intensity. For instance, a clear difference between m6A and adenosine is evident when m6A occupies the fourth position in the 5mer (e.g., "CGGAC"), but this difference is less pronounced when m6A is in the second position (e.g., "GACTT").

To illustrate the benefits of single-molecule m6A estimation, we present an example gene (ENSMUSG00000003153) from the mESC dataset used in site-level m6A identification. Figure 4C shows a heatmap of highly modified genomic locations (modification rate > 0.1), where rows represent reads and columns represent genomic locations. Biclustering reveals six clusters of reads and three clusters of genomic locations.

The heatmap in Figure 4C suggests heterogeneity in m6A modification patterns across different reads of the same gene. Biclustering reveals that modifications at g6 are specific to cluster C4, g7 to cluster C5, and g8 to cluster C6, while the first five genomic locations (g1 to g5) show similar modification patterns across all reads. Additionally, high modification rates are observed at the 3rd and 5th positions across the majority of reads. These results suggest that m6A modification

patterns can vary significantly even within a single gene. This observation highlights the complexity of RNA modification regulation and underscores the importance of single-molecule resolution for understanding RNA function at a finer scale.

Discussion

One of the main computational challenges in direct RNA sequencing (DRS) lies in accurately segmenting the raw current signal. We developed a segmentation algorithm that leverages the jiggling property in the physical process of DRS, resulting in cleaner current signals for m6A identification at both the site and single-molecule levels. Our results demonstrate that SegPore's segmentation enables clear differentiation between m6A-modified and unmodified adenosines. We believe that the de-noised current signals will be beneficial for other downstream tasks, such as the estimation of m5C, pseudouridine, and other RNA modifications. Nevertheless, several open questions remain for future research. In SegPore, we assume a drastic change between two consecutive 5mers, which may hold for 5mers with large difference in their current baselines but may not hold for those with small difference. As with other RNA modification estimation methods, SegPore can be affected by misalignment errors, particularly when the baseline signals of adjacent k-mers are similar. These cases may lead to spurious bimodal signal distributions and require careful interpretation. Another key question concerns the physical interpretation of the derived base blocks. Ideally, one base block would correspond to a single 5mer, but in practice, multiple base blocks often align with one 5mer. We hypothesize that the HHMM may segment a 5mer's current signal into multiple base blocks, where the 5mer oscillates between different sub-states, each characterized by distinct baselines.

Currently, SegPore models only the modification state of the central nucleotide within the 5mer. However, modifications at other positions may also affect the signal, as shown in **Figure 4B**. Therefore, introducing multiple states for a 5mer to account for modifications at multiple positions within the same 5mer could help to improve the performance of the model. This approach, however, would significantly increase model complexity—introducing two states per location results in $2^5 = 32$ modification states per 5mer, a challenge for simple GMMs. It remains to be investigated how to model multiple modification states properly to improve the performance in RNA modification estimation. While the current two-state assumption simplifies the model, it has proven effective, as demonstrated by improved performance on m6A benchmarks at both site and single-molecule levels.

A key advantage of SegPore is its interpretability, which sets it apart from DNN-based methods. SegPore provides a clearer understanding of RNA modification predictions. A limitation of DNN-based approaches is that they often struggle to differentiate between m6A and normal adenosine signals, as their intensity levels are quite similar (**Fig. 4B**, Supplementary Fig. S3). This poses a challenge when applying DNN-based methods to new datasets without short read sequencing-based ground truth. In such cases, it is difficult for researchers to confidently determine whether a predicted m6A modification is genuine. SegPore codes current intensity levels for different 5mers in a parameter table, where unmodified and modified 5mers are modeled using two Gaussian distributions. One can generally observe a clear difference in the intensity levels between 5mers with an m6A and those with a normal adenosine, which makes it easier for a researcher to interpret whether a predicted m6A site is genuine (see Supplementary Fig. S5). A challenge is the relatively small number of 5mers that show significant changes in their modification states. To improve accuracy, larger training datasets and expanding the scope to 7mers or 9mers could help capture more context, potentially revealing more significant baseline changes.

The limited improvement of SegPore combined with m6Anet over Nanopolish+m6Anet in bulk *in vivo* analysis (**Fig. 3**) may be explained by several factors: potential alignment inaccuracies due to pseudogenes or transcript isoforms, the complexity of *in vivo* datasets containing additional

RNA modifications (e.g., m5C, m7G) affecting signal baselines, and the fact that m6Anet is specifically trained on events produced by Nanopolish rather than SegPore. Additionally, the lack of a modification-free control (*in vitro* transcribed) sample in the benchmark dataset makes it difficult to establish true baselines for each k-mer. Despite these limitations, SegPore demonstrates clear improvement in single-molecule m6A identification in IVT data (**Fig. 4** [↗](#)), suggesting it is particularly well suited for *in vitro* transcription data analysis.

Although SegPore provides clear interpretability, there is potential to explore DNN-based models that can directly leverage SegPore's segmentation results. Currently, m6Anet computes features (e.g., mean and standard deviation) from raw signal segments, which are then fed into a neural network for m6A prediction. However, a more direct approach—where raw signal segments are used as input to a DNN—could allow for the extraction of more intricate features that may exist within the signal but are currently underexplored. These high-order features might capture subtle aspects of the raw signal, leading to improved m6A estimation. Since SegPore currently models only the mean and standard deviation, further work involving advanced DNNs could extend beyond this, uncovering finer patterns in the signal that traditional statistical models might miss.

Computation speed is also a concern when processing fast5 files. We addressed this by implementing a GPU-accelerated inference algorithm in SegPore, resulting in a 10-to 20-fold speedup compared to the CPU-based version. We believe that the GPU-implementation will unlock the full potential of SegPore for a wider range of downstream tasks and larger datasets. SegPore's running times on datasets of varying sizes, using a single NVIDIA DGX-1 V100 GPU and one CPU core, are provided in Supplementary Fig. S6.

In summary, we developed a novel software SegPore that considered the conformation changes of motor protein to segment raw current signal of Nanopore direct RNA sequencing. SegPore effectively masks out noise in the raw signal, leading to improved m6A identification at both site and single-molecule levels.

Acknowledgements

We would like to thank Prof. Zhijie Tan from Wuhan University for a useful discussion about the molecule dynamics of Nanopore sequencing, Dr. Dan Zhang from Sichuan University for helpful tutorials about Nanopore analysis workflows. We also thank Prof. Luo Guanzheng for sharing the m6A benchmark baseline results. G.C. and L.C. acknowledge the computational resources provided by the Aalto Science-IT project.

Additional information

Data availability

The data utilized in this study are obtained from publicly available repositories. Details regarding the accession number and data processing can be found in Methods. The resulting data from this work are provided in supplementary files. The source code is hosted on GitHub (<https://github.com/guangzhaocs/SegPore> [↗](#)).

Author contributions

Guangzhao Cheng: Formal analysis, Methodology, Validation, Writing—original draft & editing. Aki Vehtari: Writing—review. Lu Cheng: Conceptualization, Formal analysis, Methodology, Writing—original draft & review.

Funding

This work was supported by Research Council of Finland grants [335858, 358086 to G.C. and L.C.].

Funding

Research Council of Finland (335858)

Research Council of Finland (358086)

Additional files

Supplementary Figures and Notes. [🔗](#)

References

1. Yankova E., Aspris D., Tzelepis K (2021) **The N6-methyladenosine RNA modification in acute myeloid leukemia** *Curr Opin Hematol* **28**:80–85 [Google Scholar](#)
2. Prieto M., Folci A., Martin S (2020) **Post-translational modifications of the Fragile X Mental Retardation Protein in neuronal function and dysfunction** *Mol Psychiatry* **25**:1688–1703 [Google Scholar](#)
3. Bellodi C., McMahon M., Contreras A., Juliano D., Kopmar N., Nakamura T., Maltby D., Burlingame A., Savage S.A., Shimamura A., Ruggero D (2013) **H/ACA small RNA dysfunctions in disease reveal key roles for noncoding RNA modifications in hematopoietic stem cell differentiation** *Cell Rep* **3**:1493–1502 [Google Scholar](#)
4. Lee H., Bao S., Qian Y., Geula S., Leslie J., Zhang C., Hanna J.H., Ding L (2019) **Stage-specific requirement for Mettl3-dependent m(6)A mRNA methylation during haematopoietic stem cell differentiation** *Nat Cell Biol* **21**:700–709 [Google Scholar](#)
5. Quin J., Sedmik J., Vukic D., Khan A., Keegan L.P., O’Connell M.A (2021) **ADAR RNA Modifications, the Epitranscriptome and Innate Immunity** *Trends Biochem Sci* **46**:758–771 [Google Scholar](#)
6. Boccaletto P., Stefaniak F., Ray A., Cappannini A., Mukherjee S., Purta E., Kurkowska M., Shirvanizadeh N., Destefanis E., Groza P., et al. (2022) **MODOMICS: a database of RNA modification pathways. 2021 update** *Nucleic Acids Res* **50**:D231–D235 [Google Scholar](#)
7. Zimna M., Dolata J., Szweykowska-Kulinska Z., Jarmolowski A (2023) **The expanding role of RNA modifications in plant RNA polymerase II transcripts: highlights and perspectives** *J Exp Bot* **74**:3975–3986 [Google Scholar](#)
8. Ohira T., Suzuki T (2024) **Transfer RNA modifications and cellular thermotolerance** *Mol Cell* **84**:94–106 [Google Scholar](#)
9. Chen A.Y., Owens M.C., Liu K.F (2023) **Coordination of RNA modifications in the brain and beyond** *Mol Psychiatry* **28**:2737–2749 [Google Scholar](#)
10. Agris P.F., Narendran A., Sarachan K., Vare V.Y.P., Eruysal E (2017) **The Importance of Being Modified: The Role of RNA Modifications in Translational Fidelity** *Enzymes* **41**:1–50 [Google Scholar](#)
11. Nance K.D., Meier J.L (2021) **Modifications in an Emergency: The Role of N1-Methylpseudouridine in COVID-19 Vaccines** *ACS Cent Sci* **7**:748–756 [Google Scholar](#)
12. Meyer K.D., Saletore Y., Zumbo P., Elemento O., Mason C.E., Jaffrey S.R (2012) **Comprehensive analysis of mRNA methylation reveals enrichment in 3’ UTRs and near stop codons** *Cell* **149**:1635–1646 [Google Scholar](#)
13. Linder B., Grozhik A.V., Olarerin-George A.O., Meydan C., Mason C.E., Jaffrey S.R (2015) **Single-nucleotide-resolution mapping of m6A and m6Am throughout the transcriptome** *Nat Methods* **12**:767–772 [Google Scholar](#)

14. Koh C.W.Q., Goh Y.T., Goh W.S.S (2019) **Atlas of quantitative single-base-resolution N(6)-methyl-adenine methylomes** *Nat Commun* **10**:5636 [Google Scholar](#)
15. Li H (2018) **Minimap2: pairwise alignment for nucleotide sequences** *Bioinformatics* **34**:3094–3100 [Google Scholar](#)
16. Loman N.J., Quick J., Simpson J.T (2015) **A complete bacterial genome assembled de novo using only nanopore sequencing data** *Nat Methods* **12**:733–735 [Google Scholar](#)
17. Simpson J.T., Workman R.E., Zuzarte P.C., David M., Dursi L.J., Timp W (2017) **Detecting DNA cytosine methylation using nanopore sequencing** *Nat Methods* **14**:407–410 [Google Scholar](#)
18. Stoiber M., Quick J., Egan R., Eun Lee J., Celniker S., Neely R.K., Loman N., Pennacchio L.A., Brown J. (2017) **De novo Identification of DNA Modifications Enabled by Genome-Guided Nanopore Signal Processing** *bioRxiv* [Google Scholar](#)
19. Hendra C., Pratanwanich P.N., Wan Y.K., Goh W.S.S., Thierry A., Goke J (2022) **Detection of m6A from direct RNA sequencing using a multiple instance learning framework** *Nat Methods* **19**:1590–1598 [Google Scholar](#)
20. Acera Mateos P., Ravindran A., Srivastava A., Woodward K., Mahmud S., Kanchi M., Guarnacci M., Xu J., et al. (2024) **Prediction of m6A and m5C at single-molecule resolution reveals a transcriptome-wide co-occurrence of RNA modifications** *Nat Commun* **15**:3899 [Google Scholar](#)
21. Pratanwanich P.N., Yao F., Chen Y., Koh C.W.Q., Wan Y.K., Hendra C., Poon P., Goh Y.T., Yap P.M.L., Chooi J.Y., et al. (2021) **Identification of differential RNA modifications from nanopore direct RNA sequencing with xPore** *Nat Biotechnol* **39**:1394–1402 [Google Scholar](#)
22. Jenjaroenpun P., Wongsurawat T., Wadley T.D., Wassenaar T.M., Liu J., Dai Q., Wanchai V., Akel N.S., Jamshidi-Parsian A., Franco A.T., et al. (2021) **Decoding the epitranscriptional landscape from native RNA sequences** *Nucleic Acids Res* **49**:e7 [Google Scholar](#)
23. Zhong Z.D., Xie Y.Y., Chen H.X., Lan Y.L., Liu X.H., Ji J.Y., Wu F., Jin L., Chen J., Mak D.W., et al. (2023) **Systematic comparison of tools used for m(6)A mapping from nanopore direct RNA sequencing** *Nat Commun* **14**:1906 [Google Scholar](#)
24. Craig J.M., Laszlo A.H., Brinkerhoff H., Derrington I.M., Noakes M.T., Nova I.C., Tickman B.I., Doering K., de Leeuw N.F., Gundlach J.H. (2017) **Revealing dynamics of helicase translocation on single-stranded DNA using high-resolution nanopore tweezers** *Proc Natl Acad Sci U S A* **114**:11932–11937 [Google Scholar](#)
25. Caldwell C.C., Spies M (2017) **Helicase SPRNTing through the nanopore** *Proc Natl Acad Sci U S A* **114**:11809–11811 [Google Scholar](#)
26. Chen Y., Davidson N.M., Wan Y.K., Patel H., Yao F., Low H.M., Hendra C., Watten L., Sim A., Sawyer C., et al. (2021) **A systematic benchmark of Nanopore long read RNA sequencing for transcript level analysis in human cell lines** *bioRxiv* [Google Scholar](#)
27. Gamaarachchi H., Lam C.W., Jayatilaka G., Samarakoon H., Simpson J.T., Smith M.A., Parameswaran S (2020) **GPU accelerated adaptive banded event alignment for rapid comparative nanopore signal analysis** *BMC Bioinformatics* **21**:343 [Google Scholar](#)

28. Kovaka S., Hook P.W., Jenike K.M., Shivakumar V., Morina L.B., Razaghi R., Timp W., Schatz M.C (2025) **Uncalled4 improves nanopore DNA and RNA modification detection via fast and accurate signal alignment** *Nat Methods* **22**:681–691 [Google Scholar](#)
29. Watson K.J., Bromley R.E., Dunning Hotopp J.C (2025) **Duplexed direct RNA sequencing protocol using polyadenylation and polyuridylation** *Microbiol Resour Announc* **14**:e0104124 [Google Scholar](#)
30. Cruciani S., Delgado-Tejedor A., Pryszcz L.P., Medina R., Llovera L., Novoa E.M (2025) **De novo basecalling of RNA modifications at single molecule and nucleotide resolution** *Genome Biol* **26**:38 [Google Scholar](#)
31. Lorenz D.A., Sathe S., Einstein J.M., Yeo G.W (2020) **Direct RNA sequencing enables m(6)A detection in endogenous transcript isoforms at base-specific resolution** *RNA* **26**:19–28 [Google Scholar](#)
32. Gao Y., Liu X., Wu B., Wang H., Xi F., Kohnen M.V., Reddy A.S.N., Gu L (2021) **Quantitative profiling of N(6)-methyladenosine at single-base resolution in stem-differentiating xylem of *Populus trichocarpa* using Nanopore direct RNA sequencing** *Genome Biol* **22**:22 [Google Scholar](#)
33. Liu H., Begik O., Lucas M.C., Ramirez J.M., Mason C.E., Wiener D., Schwartz S., Mattick J.S., Smith M.A., Novoa E.M (2019) **Accurate detection of m(6)A RNA modifications in native RNA sequences** *Nat Commun* **10**:4079 [Google Scholar](#)
34. Kortel N., Ruckle C., Zhou Y., Busch A., Hoch-Kraft P., Sutandy F.X.R., Haase J., Pradhan M., Musheev M., Ostareck D., et al. (2021) **Deep and accurate detection of m6A RNA modifications using miCLIP2 and m6Aboost machine learning** *Nucleic Acids Res* **49**:e92 [Google Scholar](#)

Author information

Guangzhao Cheng

Department of Computer Science, Aalto University, Espoo, Finland
ORCID iD: [0009-0008-6160-3637](#)

Aki Vehtari

Department of Computer Science, Aalto University, Espoo, Finland
ORCID iD: [0000-0003-2164-9469](#)

Lu Cheng

Department of Computer Science, Aalto University, Espoo, Finland, Institute of Biomedicine, University of Eastern Finland, Kuopio, Finland
ORCID iD: [0000-0002-6391-2360](#)

For correspondence: lu.cheng@uef.fi

Editors

Reviewing Editor

Nicolas Altemose

Stanford University, United States of America

Senior Editor

Alan Moses

University of Toronto, Toronto, Canada

Reviewer #1 (Public review):

Summary:

In this manuscript, the authors describe a new computational method (SegPore), which segments the raw signal from nanopore direct RNA-Seq data to improve the identification of RNA modifications. In addition to signal segmentation, SegPore includes a Gaussian Mixture Model approach to differentiate modified and unmodified bases. SegPore uses Nanopolish to define a first segmentation, which is then refined into base and transition blocks. SegPore also includes a modification prediction model that is included in the output. The authors evaluate the segmentation in comparison to Nanopolish and Tombo (RNA002) as well as f5c and Uncalled 4 (RNA004), and they evaluate the impact on m6A RNA modification detection using data with known m6A sites. In comparison to existing methods, SegPore appears to improve the ability to detect m6A, suggesting that this approach could be used to improve the analysis of direct RNA-Seq data.

Strengths:

SegPore address an important problem (signal data segmentation). By refining the signal into transition and base blocks, noise appears to be reduced, leading to improved m6A identification at the site level as well as for single read predictions. The authors provide a fully documented implementation, including a GPU version that reduces run time. The authors provide a detailed methods description, and the approach to refine segments appears to be new.

Weaknesses:

The authors show that SegPore reduces noise compared to other methods, however the improvement in accuracy appears to be relatively small for the task of identifying m6A. To run SegPore, the GPU version is essential, which could limit the application of this method in practice.

<https://doi.org/10.7554/eLife.104618.2.sa3>

Reviewer #2 (Public review):

Summary:

The work seeks to improve detection of RNA m6A modifications using Nanopore sequencing through improvements in raw data analysis. These improvements are said to be in the segmentation of the raw data, although the work appears to position the alignment of raw data to the reference sequence and some further processing as part of the segmentation, and result statistics are mostly shown on the 'data-assigned-to-kmer' level.

As such, the title, abstract and introduction stating the improvement of just the 'segmentation'

does not seem to match the work the manuscript actually presents, as the wording seems a bit too limited for the work involved.

The work itself shows minor improvements in m6Anet when replacing Nanopolish' eventalign with this new approach, but clear improvements in the distributions of data assigned per kmer. However, these assignments were improved well enough to enable m6A calling from them directly, both at site-level and at read-level.

A large part of the improvements shown appear to stem from the addition of extra, non-base/kmer specific, states in the segmentation/assignment of the raw data, removing a significant portion of what can be considered technical noise for further analysis. Previous methods enforced assignment of (almost) all raw data, forcing a technically optimal alignment that may lead to suboptimal results in downstream processing as datapoints could be assigned to neighbouring kmers instead, while random noise that is assigned to the correct kmer may also lead to errors in modification detection.

For an optimal alignment between the raw signal and the reference sequence, this approach may yield improvements for downstream processing using other tools.

Additionally, the GMM used for calling the m6A modifications provides a useful, simple and understandable logic to explain the reason a modification was called, as opposed to the black models that are nowadays often employed for these types of tasks.

Weaknesses:

The manuscript suggests the eventalign results are improved compared to Nanopolish. While this is believably shown to be true (Table 1), the effect on the use case presented, downstream differentiation between modified and unmodified status on a base/kmer, is likely limited for during downstream modification calling the noisy distributions are often 'good enough'. E.g. Nanopolish uses the main segmentation+alignment for a first alignment and follows up with a form of targeted local realignment/HMM test for modification calling (and for training too), decreasing the need for the near-perfect segmentation+alignment this work attempts to provide. Any tool applying a similar strategy probably largely negates the problems this manuscript aims to improve upon. Should a use-case come up where this downstream optimisation is not an option, SegPore might provide the necessary improvements in raw data alignment.

Appraisal:

The authors have shown their methods ability to identify noise in the raw signal and remove their values from the segmentation and alignment, reducing its influences for further analyses. Figures directly comparing the values per kmer do show a visibly improved assignment of raw data per kmer. As a replacement for Nanopolish' eventalign it seems to have a rather limited, but improved effect, on m6Anet results. At the single read level modification modification calling this work does appear to improve upon CHEUI.

Impact:

With the current developments for Nanopore based modification calling largely focusing on Artificial Intelligence, Neural Networks and the likes, improvements made in interpretable approaches provide an important alternative that enables deeper understanding of the data rather than providing a tool that plainly answers the question of whether a base is modified or not, without further explanation. The work presented is best viewed in context of a workflow where one aims to get an optimal alignment between raw signal data and the reference base sequence for further processing. For example, as presented, as a possible replacement for Nanopolish' eventalign. Here it might enable data exploration and downstream modification calling without the need for local realignments or other approaches that re-consider the distribution of raw data around the target motif, such as a 'local' Hidden Markov Model or

Neural Networks. These possibilities are useful for a deeper understanding of the data and further tool development for modification detection works beyond m6A calling.

<https://doi.org/10.7554/eLife.104618.2.sa2>

Reviewer #3 (Public review):

Summary:

Nucleotide modifications are important regulators of biological function, however, until recently, their study has been limited by the availability of appropriate analytical methods. Oxford Nanopore direct RNA sequencing preserves nucleotide modifications, permitting their study, however many different nucleotide modifications lack an available base-caller to accurately identify them. Furthermore, existing tools are computationally intensive, and their results can be difficult to interpret.

Cheng et al. present SegPore, a method designed to improve the segmentation of direct RNA sequencing data and boost the accuracy of modified base detection.

Strengths:

This method is well described and has been benchmarked against a range of publicly available base callers that have been designed to detect modified nucleotides.

Weaknesses:

However, the manuscript has a significant drawback in its current version. The most recent nanopore RNA base callers can distinguish between different ribonucleotide modifications, however, SegPore has not been benchmarked against these models.

The manuscript would be strengthened by benchmarking against the rna004_130bps_hac@v5.1.0 and rna004_130bps_sup@v5.1.0 dorado models, which are reported to detect m5C, m6A_DRACH, inosine_m6A and PseU.

A clear demonstration that SegPore also outperforms the newer RNA base caller models will confirm the utility of this method.

<https://doi.org/10.7554/eLife.104618.2.sa1>

Author response:

The following is the authors' response to the original reviews

We thank all the reviewers for their constructive comments. We have carefully considered your feedback and revised the manuscript accordingly. The major concern raised was the applicability of SegPore to the RNA004 dataset. To address this, we compared SegPore with f5c and Uncalled4 on RNA004, and found that SegPore demonstrated improved performance, as shown in Table 2 of the revised manuscript.

Following the reviewers' recommendations, we updated Figures 3 and 4. Additionally, we added one table and three supplementary figures to the revised manuscript:

- Table 2: Segmentation benchmark on RNA004 data
- Supplementary Figure S4: RNA translocation hypothesis illustrated on RNA004 data

- Supplementary Figure S5: Illustration of Nanopolish raw signal segmentation with eventalign results
- Supplementary Figure S6: Running time of SegPore on datasets of varying sizes

Below, we provide a point-by-point response to your comments.

Reviewer #1 (Public review):

Summary:

In this manuscript, the authors describe a new computational method (SegPore), which segments the raw signal from nanopore-direct RNA-Seq data to improve the identification of RNA modifications. In addition to signal segmentation, SegPore includes a Gaussian Mixture Model approach to differentiate modified and unmodified bases. SegPore uses Nanopolish to define a first segmentation, which is then refined into base and transition blocks. SegPore also includes a modification prediction model that is included in the output. The authors evaluate the segmentation in comparison to Nanopolish and Tombo, and they evaluate the impact on m6A RNA modification detection using data with known m6A sites. In comparison to existing methods, SegPore appears to improve the ability to detect m6A, suggesting that this approach could be used to improve the analysis of direct RNA-Seq data.

Strengths:

SegPore addresses an important problem (signal data segmentation). By refining the signal into transition and base blocks, noise appears to be reduced, leading to improved m6A identification at the site level as well as for single-read predictions. The authors provide a fully documented implementation, including a GPU version that reduces run time. The authors provide a detailed methods description, and the approach to refine segments appears to be new.

Weaknesses:

In addition to Nanopolish and Tombo, f5c and Uncalled4 can also be used for segmentation, however, the comparison to these methods is not shown.

The method was only applied to data from the RNA002 direct RNA-Sequencing version, which is not available anymore, currently, it remains unclear if the methods still work on RNA004.

Thank you for your comments.

To clarify the background, there are two kits for Nanopore direct RNA sequencing: RNA002 (the older version) and RNA004 (the newer version). Oxford Nanopore Technologies (ONT) introduced the RNA004 kit in early 2024 and has since discontinued RNA002. Consequently, most public datasets are based on RNA002, with relatively few available for RNA004 (as of 30 June 2025).

Nanopolish and Tombo were developed for raw signal segmentation and alignment using RNA002 data, whereas f5c and Uncalled4 are the only two software supporting RNA004 data. Since the development of SegPore began in January 2022, we initially focused on RNA002 due to its data availability. Accordingly, our original comparisons were made against Nanopolish and Tombo using RNA002 data.

We have now updated SegPore to support RNA004 and compared its performance against f5c and Uncalled4 on three public RNA004 datasets.

As shown in Table 2 of the revised manuscript, SegPore outperforms both f5c and Uncalled4 in raw signal segmentation. Moreover, the jiggling translocation hypothesis underlying SegPore is further supported, as shown in Supplementary Figure S4.

| *The overall improvement in accuracy appears to be relatively small.*

Thank you for the comment.

We understand that the improvements shown in Tables 1 and 2 may appear modest at first glance due to the small differences in the reported standard deviation (std) values. However, even small absolute changes in std can correspond to substantial relative reductions in noise, especially when the total variance is low.

To better quantify the improvement, we assume that approximately 20% of the std for Nanopolish, Tombo, f5c, and Uncalled4 arises from noise. Using this assumption, we calculate the relative noise reduction rate of SegPore as follows:

Noise reduction rate = (baseline std – SegPore std) / (0.2 × baseline std)

Based on this formula, the average noise reduction rates across all datasets are:

- SegPore vs Nanopolish: 49.52%
- SegPore vs Tombo: 167.80%
- SegPore vs f5c: 9.44%
- SegPore vs Uncalled4: 136.70%

These results demonstrate that SegPore can reduce the noise level by at least 9% given a noise level of 20%, which we consider a meaningful improvement for downstream tasks, such as base modification detection and signal interpretation. The high noise reduction rates observed in Tombo and Uncalled4 (over 100%) suggest that their actual noise proportion may be higher than our 20% assumption.

We acknowledge that this 20% noise level assumption is an approximation. Our intention is to illustrate that SegPore provides measurable improvements in relative terms, even when absolute differences appear small.

| *The run time and resources that are required to run SegPore are not shown, however, it appears that the GPU version is essential, which could limit the application of this method in practice.*

Thank you for your comment.

Detailed instructions for running SegPore are provided in github (<https://github.com/guangzhaocs/SegPore>). Regarding computational resources, SegPore currently requires one CPU core and one Nvidia GPU to perform the segmentation task efficiently.

We present SegPore's runtime for typical datasets in Supplementary Figure S6 in the revised manuscript. For a typical 1 GB fast5 file, the segmentation takes approximately 9.4 hours using a single NVIDIA DGX-1 V100 GPU and one CPU core.

Currently, GPU acceleration is essential to achieve practical runtimes with SegPore. We acknowledge that this requirement may limit accessibility in some environments. To address this, we are actively working on a full C++ implementation of SegPore that will support CPU-

only execution. While development is ongoing, we aim to release this version in a future update.

Reviewer #2 (Public review):

Summary:

The work seeks to improve the detection of RNA m6A modifications using Nanopore sequencing through improvements in raw data analysis. These improvements are said to be in the segmentation of the raw data, although the work appears to position the alignment of raw data to the reference sequence and some further processing as part of the segmentation, and result statistics are mostly shown on the 'data-assigned-to-kmer' level.

As such, the title, abstract, and introduction stating the improvement of just the 'segmentation' does not seem to match the work the manuscript actually presents, as the wording seems a bit too limited for the work involved.

The work itself shows minor improvements in m6Anet when replacing Nanopolish eventalign with this new approach, but clear improvements in the distributions of data assigned per kmer. However, these assignments were improved well enough to enable m6A calling from them directly, both at site-level and at read-level.

Strengths:

A large part of the improvements shown appear to stem from the addition of extra, non-base/kmer specific, states in the segmentation/assignment of the raw data, removing a significant portion of what can be considered technical noise for further analysis. Previous methods enforced the assignment of all raw data, forcing a technically optimal alignment that may lead to suboptimal results in downstream processing as data points could be assigned to neighbouring kmers instead, while random noise that is assigned to the correct kmer may also lead to errors in modification detection.

For an optimal alignment between the raw signal and the reference sequence, this approach may yield improvements for downstream processing using other tools. Additionally, the GMM used for calling the m6A modifications provides a useful, simple, and understandable logic to explain the reason a modification was called, as opposed to the black models that are nowadays often employed for these types of tasks.

Weaknesses:

The work seems limited in applicability largely due to the focus on the R9's 5mer models. The R9 flow cells are phased out and not available to buy anymore. Instead, the R10 flow cells with larger kmer models are the new standard, and the applicability of this tool on such data is not shown. We may expect similar behaviour from the raw sequencing data where the noise and transition states are still helpful, but the increased kmer size introduces a large amount of extra computing required to process data and without knowledge of how SegPore scales, it is difficult to tell how useful it will really be. The discussion suggests possible accuracy improvements moving to 7mers or 9mers, but no reason why this was not attempted.

Thank you for pointing out this important limitation. Please refer to our response to Point 1 of Reviewer 1 for SegPore's performance on RNA004 data. Notably, the jiggling behavior is also observed in RNA004 data, and SegPore achieves better performance than both f5c and Uncalled4.

The increased k-mer size in RNA004 affects only the training phase of SegPore (refer to Supplementary Note 1, Figure 5 for details on the training and testing phases). Once the baseline means and standard deviations for each k-mer are established, applying SegPore to RNA004 data proceeds similarly to RNA002. This is because each k-mer in the reference sequence has, at most, two states (modified and unmodified). While the larger k-mer size increases the size of the parameter table, it does not increase the computational complexity during segmentation. Although estimating the initial k-mer parameter table requires significant time and effort on our part, it does not affect the runtime for end users applying SegPore to RNA004 data.

Extending SegPore from 5-mers to 7-mers or 9-mers for RNA002 data would require substantial effort to retrain the model and generate sufficient training data. Additionally, such an extension would make SegPore's output incompatible with widely used upstream and downstream tools such as Nanopolish and m6Anet, complicating integration and comparison. For these reasons, we leave this extension for future work.

The manuscript suggests the eventalign results are improved compared to Nanopolish. While this is believably shown to be true (Table 1), the effect on the use case presented, downstream differentiation between modified and unmodified status on a base/kmer, is likely limited as during actual modification calling the noisy distributions are usually 'good enough', and not skewed significantly in one direction to really affect the results too terribly.

Thank you for your comment. While current state-of-the-art (SOTA) methods perform well on benchmark datasets, there remains significant room for improvement. Most SOTA evaluations are based on limited datasets, primarily covering DRACH motifs in human and mouse transcriptomes. However, m6A modifications can also occur in non-DRACH motifs, where current models may underperform. Additionally, other RNA modifications—such as pseudouridine, inosine, and m5C—are less studied, and their detection may benefit from improved signal modeling.

We would also like to emphasize that raw signal segmentation and RNA modification detection are distinct tasks. SegPore focuses on the former, providing a cleaner, more interpretable signal that can serve as a foundation for downstream tasks. Improved segmentation may facilitate the development of more accurate RNA modification detection algorithms by the community.

Scientific progress often builds incrementally through targeted improvements to foundational components. We believe that enhancing signal segmentation, as SegPore does, contributes meaningfully to the broader field—the full impact will become clearer as the tool is adopted into more complex workflows.

Furthermore, looking at alternative approaches where this kind of segmentation could be applied, Nanopolish uses the main segmentation+alignment for a first alignment and follows up with a form of targeted local realignment/HMM test for modification calling (and for training too), decreasing the need for the near-perfect segmentation+alignment this work attempts to provide. Any tool applying a similar strategy probably largely negates the problems this manuscript aims to improve upon.

We thank the reviewer for this insightful comment.

To clarify, Nanopolish provides three independent commands: polya, eventalign, and call-methylation.

- The polya command identifies the adapter, poly(A) tail, and transcript region in the raw signal.
- The eventalign command aligns the raw signal to a reference sequence, assigning a signal segment to individual k-mers in the reference.
- The call-methylation command detects methylated bases from DNA sequencing data.

The eventalign command corresponds to “the main segmentation+alignment for a first alignment,” while call-methylation corresponds to “a form of targeted local realignment/HMM test for modification calling,” as mentioned in the reviewer’s comment. SegPore’s segmentation is similar in purpose to Nanopolish’s eventalign, while its RNA modification estimation component is similar in concept to Nanopolish’s call-methylation.

We agree the general idea may appear similar, but the implementations are entirely different. Importantly, Nanopolish’s call-methylation is designed for DNA sequencing data, and its models are not trained to recognize RNA modifications. This means they address distinct research questions and cannot be directly compared on the same RNA modification estimation task. However, it is valid to compare them on the segmentation task, where SegPore exhibits better performance (Table 1).

We infer the reviewer may suggest that because m6Anet is a deep neural network capable of learning from noisy input, the benefit of more accurate segmentation (such as that provided by SegPore) might be limited. This concern may arise from the limited improvement of SegPore+m6Anet over Nanopolish+m6Anet in bulk analysis (Figure 3). Several factors may contribute to this observation:

- (i) For reads aligned to the same gene in the in vivo data, alignment may be inaccurate due to pseudogenes or transcript isoforms.
- (ii) The in vivo benchmark data are inherently more complex than in vitro datasets and may contain additional modifications (e.g., m5C, m7G), which can confound m6A calling by altering the signal baselines of k-mers.
- (iii) m6Anet is trained on events produced by Nanopolish and may not be optimal for SegPore-derived events.
- (iv) The benchmark dataset lacks a modification-free (IVT) control sample, making it difficult to establish a true baseline for each k-mer.

In the IVT data (Figure 4), SegPore shows a clear improvement in single-molecule m6A identification, with a 3~4% gain in both ROC-AUC and PR-AUC. This demonstrates SegPore’s practical benefit for applications requiring higher sensitivity at the molecule level.

As noted earlier, SegPore’s contribution lies in denoising and improving the accuracy of raw signal segmentation, which is a foundational step in many downstream analyses. While it may not yet lead to a dramatic improvement in all applications, it already provides valuable insights into the sequencing process (e.g., cleaner signal profiles in Figure 4) and enables measurable gains in modification detection at the single-read level. We believe SegPore lays the groundwork for developing more accurate and generalizable RNA modification detection tools beyond m6A.

We have also added the following sentence in the discussion to highlight SegPore’s limited performance in bulk analysis:

“The limited improvement of SegPore combined with m6Anet over Nanopolish+m6Anet in bulk in vivo analysis (Figure 3) may be explained by several factors: potential alignment

inaccuracies due to pseudogenes or transcript isoforms, the complexity of in vivo datasets containing additional RNA modifications (e.g., m5C, m7G) affecting signal baselines, and the fact that m6Anet is specifically trained on events produced by Nanopolish rather than SegPore. Additionally, the lack of a modification-free control (in vitro transcribed) sample in the benchmark dataset makes it difficult to establish true baselines for each k-mer. Despite these limitations, SegPore demonstrates clear improvement in single-molecule m6A identification in IVT data (Figure 4), suggesting it is particularly well suited for in vitro transcription data analysis.”

Finally, in the segmentation/alignment comparison to Nanopolish, the latter was not fitted(/trained) on the same data but appears to use the pre-trained model it comes with. For the sake of comparing segmentation/alignment quality directly, fitting Nanopolish on the same data used for SegPore could remove the influences of using different training datasets and focus on differences stemming from the algorithm itself.

In the segmentation benchmark (Table 1), SegPore uses the fixed 5-mer parameter table provided by ONT. The hyperparameters of the HHMM are also fixed and not estimated from the raw signal data being segmented. Only in the m6A modification task, SegPore does perform re-estimation of the baselines for the modified and unmodified states of k-mers. Therefore, the comparison with Nanopolish is fair, as both tools rely on pre-defined models during segmentation.

Appraisal:

The authors have shown their method's ability to identify noise in the raw signal and remove their values from the segmentation and alignment, reducing its influences for further analyses. Figures directly comparing the values per kmer do show a visibly improved assignment of raw data per kmer. As a replacement for Nanopolish eventalign it seems to have a rather limited, but improved effect, on m6Anet results. At the single read level modification modification calling this work does appear to improve upon CHEUI.

Impact:

With the current developments for Nanopore-based modification largely focusing on Artificial Intelligence, Neural Networks, and the like, improvements made in interpretable approaches provide an important alternative that enables a deeper understanding of the data rather than providing a tool that plainly answers the question of whether a base is modified or not, without further explanation. The work presented is best viewed in the context of a workflow where one aims to get an optimal alignment between raw signal data and the reference base sequence for further processing. For example, as presented, as a possible replacement for Nanopolish eventalign. Here it might enable data exploration and downstream modification calling without the need for local realignments or other approaches that re-consider the distribution of raw data around the target motif, such as a 'local' Hidden Markov Model or Neural Networks. These possibilities are useful for a deeper understanding of the data and further tool development for modification detection works beyond m6A calling.

Reviewer #3 (Public review):

Summary:

Nucleotide modifications are important regulators of biological function, however, until recently, their study has been limited by the availability of appropriate analytical methods. Oxford Nanopore direct RNA sequencing preserves nucleotide modifications, permitting their study, however, many different nucleotide modifications lack an

available base-caller to accurately identify them. Furthermore, existing tools are computationally intensive, and their results can be difficult to interpret.

Cheng et al. present SegPore, a method designed to improve the segmentation of direct RNA sequencing data and boost the accuracy of modified base detection.

Strengths:

This method is well-described and has been benchmarked against a range of publicly available base callers that have been designed to detect modified nucleotides.

Weaknesses:

However, the manuscript has a significant drawback in its current version. The most recent nanopore RNA base callers can distinguish between different ribonucleotide modifications, however, SegPore has not been benchmarked against these models.

I recommend that re-submission of the manuscript that includes benchmarking against the rna004_130bps_hac@v5.1.0 and rna004_130bps_sup@v5.1.0 dorado models, which are reported to detect m5C, m6A_DRACH, inosine_m6A and PseU.

A clear demonstration that SegPore also outperforms the newer RNA base caller models will confirm the utility of this method.

Thank you for highlighting this important limitation. While Dorado, the new ONT basecaller, is publicly available and supports modification-aware basecalling, suitable public datasets for benchmarking m5C, inosine, m6A, and PseU detection on RNA004 are currently lacking. Dorado's modification-aware models are trained on ONT's internal data, which is not publicly released. Therefore, it is not currently feasible to evaluate or directly compare SegPore's performance against Dorado for m5C, inosine, m6A, and PseU detection.

We would also like to emphasize that SegPore's main contribution lies in raw signal segmentation, which is an upstream task in the RNA modification detection pipeline. To assess its performance in this context, we benchmarked SegPore against f5c and Uncalled4 on public RNA004 datasets for segmentation quality. Please refer to our response to Point 1 of Reviewer 1 for details.

Our results show that the characteristic "jiggling" behavior is also observed in RNA004 data (Supplementary Figure S4), and SegPore achieves better segmentation performance than both f5c and Uncalled4 (Table 2).

Recommendations for the authors:

Reviewing Editor:

Please note that we also received the following comments on the submission, which we encourage you to take into account:

took a look at the work and for what I saw it only mentions/uses RNA002 chemistry, which is deprecated, effectively making this software unusable by anyone any more, as RNA002 is not commercially available. While the results seem promising, the authors need to show that it would work for RNA004. Notably, there is an alternative software for resquiggling for RNA004 (not Tombo or Nanopolish, but the GPU-accelerated version of Nanopolish (f5C), which does support RNA004. Therefore, they need to show that SegPore works for RNA004, because otherwise it is pointless to see that this method works better than others if it does not support current sequencing chemistries and only works for deprecated chemistries, and people will keep using f5C because its the only one that currently works for RNA004. Alternatively, if there would be biological insights won from

the method, one could justify not implementing it in RNA004, but in this case, RNA002 is deprecated since March 2024, and the paper is purely methodological.

Thank you for the comment. We agree that support for current sequencing chemistries is essential for practical utility. While SegPore was initially developed and benchmarked on RNA002 due to the availability of public data, we have now extended SegPore to support RNA004 chemistry.

To address this concern, we performed a benchmark comparison using public RNA004 datasets against tools specifically designed for RNA004, including f5c and Uncalled4. Please refer to our response to Point 1 of Reviewer 1 for details. The results show that SegPore consistently outperforms f5c and Uncalled4 in segmentation accuracy on RNA004 data.

Reviewer #2 (Recommendations for the authors):

Various statements are made throughout the text that require further explanation, which might actually be defined in more detail elsewhere sometimes but are simply hard to find in the current form.

(1) Page 2, "In this technique, five nucleotides (5mers) reside in the nanopore at a time, and each 5mer generates a characteristic current signal based on its unique sequence and chemical properties (16)."

5mer? Still on R9 or just ignoring longer range influences, relevant? It is indeed a R9.4 model from ONT.

Thank you for the observation. We apologize for the confusion and have clarified the relevant paragraph to indicate that the method is developed for RNA002 data by default. Specifically, we have added the following sentence:

"Two versions of the direct RNA sequencing (DRS) kits are available: RNA002 and RNA004. Unless otherwise specified, this study focuses on RNA002 data."

(2) Page 3, "Employ models like Hidden Markov Models (HMM) to segment the signal, but they are prone to noise and inaccuracies."

That's the alignment/calling part, not the segmentation?

Thank you for the comment. We apologize for the confusion. To clarify the distinction between segmentation and alignment, we added a new paragraph before the one in question to explain the general workflow of Nanopore DRS data analysis and to clearly define the task of segmentation. The added text reads:

"The general workflow of Nanopore direct RNA sequencing (DRS) data analysis is as follows. First, the raw electrical signal from a read is basecalled using tools such as Guppy or Dorado, which produce the nucleotide sequence of the RNA molecule. However, these basecalled sequences do not include the precise start and end positions of each ribonucleotide (or k-mer) in the signal. Because basecalling errors are common, the sequences are typically mapped to a reference genome or transcriptome using minimap2 to recover the correct reference sequence. Next, tools such as Nanopolish and Tombo align the raw signal to the reference sequence to determine which portion of the signal corresponds to each k-mer. We define this process as the segmentation task, referred to as "eventalign" in Nanopolish. Based on this alignment, Nanopolish extracts various features—such as the start and end positions, mean, and standard deviation of the signal segment corresponding to a k-mer. This signal segment or its derived features is referred to as an "event" in Nanopolish."

We also revised the following paragraph describing SegPore to more clearly contrast its approach:

“In SegPore, we first segment the raw signal into small fragments using a Hierarchical Hidden Markov Model (HHMM), where each fragment corresponds to a sub-state of a k-mer. Unlike Nanopolish and Tombo, which directly align the raw signal to the reference sequence, SegPore aligns the mean values of these small fragments to the reference. After alignment, we concatenate all fragments that map to the same k-mer into a larger segment, analogous to the "eventalign" output in Nanopolish. For RNA modification estimation, we use only the mean signal value of each reconstructed event.”

We hope this revision clarifies the difference between segmentation and alignment in the context of our method and resolves the reviewer’s concern.

(3) Page 4, Figure 1, “These segments are then aligned with the 5mer list of the reference sequence fragment using a full/partial alignment algorithm, based on a 5mer parameter table. For example, A_j denotes the base “A” at the j-th position on the reference.”

I think I do understand the meaning, but I do not understand the relevance of the A_j bit in the last sentence. What is it used for?

When aligning the segments (output from Step 2) to the reference sequence in Step 3, it is possible for multiple segments to align to the same k-mer. This can occur particularly when the reference contains consecutive identical bases, such as multiple adenines (A). For example, as shown in Fig. 1A, Step 3, the first two segments (μ_1 and μ_2) are aligned to the first 'A' in the reference sequence, while the third segment is aligned to the second 'A'. In this case, the reference sequence AACTGGTTTC...GTC, which contains exactly two consecutive 'A's at the start. This notation helps to disambiguate segment alignment in regions with repeated bases.

Additionally, this figure and its subscript include mapping with Guppy and Minimap2 but do not mention Nanopolish at all, while that seems an equally important step in the preprocessing (pg5). As such it is difficult to understand the role Nanopolish exactly plays. It's also not mentioned explicitly in the SegPore Workflow on pg15, perhaps it's part of step 1 there?

We thank the reviewer for pointing this out. We apologize for the confusion. As mentioned in the public response to point 3 of Reviewer 2, SegPore uses Nanopolish to identify the poly(A) tail and transcript regions from the raw signal. SegPore then performs segmentation and alignment on the transcript portion only. This step is indeed part of Step 1 in the preprocessing workflow, as described in Supplementary Note 1, Section 3.

To clarify this in the main text, we have updated the preprocessing paragraph on page 6 to explicitly describe the role of Nanopolish:

“We begin by performing basecalling on the input fast5 file using Guppy, which converts the raw signal data into ribonucleotide sequences. Next, we align the basecalled sequences to the reference genome using Minimap2, generating a mapping between the reads and the reference sequences. Nanopolish provides two independent commands: "polya" and "eventalign". The "polya" command identifies the adapter, poly(A) tail, and transcript region in the raw signal, which we refer to as the poly(A) detection results. The raw signal segment corresponding to the poly(A) tail is used to standardize the raw signal for each read. The "eventalign" command aligns the raw signal to a reference sequence, assigning a signal segment to individual k-mers in the reference. It also computes summary statistics (e.g., mean, standard deviation) from the signal segment for each k-mer. Each k-mer together with its corresponding signal features is termed an event. These event features are then passed

into downstream tools such as m6Anet and CHEUI for RNA modification detection. For full transcriptome analysis (Figure 3), we extract the aligned raw signal segment and reference sequence segment from Nanopolish's events for each read by using the first and last events as start and end points. For in vitro transcription (IVT) data with a known reference sequence (Figure 4), we extract the raw signal segment corresponding to the transcript region for each input read based on Nanopolish's poly(A) detection results."

Additionally, we revised the legend of Figure 1A to explicitly include Nanopolish in step 1 as follows:

"The raw current signal fragments are paired with the corresponding reference RNA sequence fragments using Nanopolish."

(4) Page 5, "The output of Step 3 is the "eventalign," which is analogous to the output generated by the Nanopolish "eventalign" command."

Naming the function of Nanopolish, the output file, and later on (pg9) the alignment of the newly introduced methods the exact same "eventalign" is very confusing.

Thank you for the helpful comment. We acknowledge the potential confusion caused by using the term "eventalign" in multiple contexts. To improve clarity, we now consistently use the term "events" to refer to the output of both Nanopolish and SegPore, rather than using "eventalign" as a noun. We also added the following sentence to Step 3 (page 6) to clearly define what an "event" refers to in our manuscript:

"An "event" refers to a segment of the raw signal that is aligned to a specific k-mer on a read, along with its associated features such as start and end positions, mean current, standard deviation, and other relevant statistics."

We have revised the text throughout the manuscript accordingly to reduce ambiguity and ensure consistent terminology.

(5) Page 5, "Once aligned, we use Nanopolish's eventalign to obtain paired raw current signal segments and the corresponding fragments of the reference sequence, providing a precise association between the raw signals and the nucleotide sequence."

I thought the new method's HHMM was supposed to output an 'eventalign' formatted file. As this is not clearly mentioned elsewhere, is this a mistake in writing? Is this workflow dependent on Nanopolish 'eventalign' function and output or not?

We apologize for the confusion. To clarify, SegPore is not dependent on Nanopolish's eventalign function for generating the final segmentation results. As described in our response to your comment point 2 and elaborated in the revised text on page 4, SegPore uses its own HHMM-based segmentation model to divide the raw signal into small fragments, each corresponding to a sub-state of a k-mer. These fragments are then aligned to the reference sequence based on their mean current values.

As explained in the revised manuscript:

"In SegPore, we first segment the raw signal into small fragments using a Hierarchical Hidden Markov Model (HHMM), where each fragment corresponds to a sub-state of a k-mer. Unlike Nanopolish and Tombo, which directly align the raw signal to the reference sequence, SegPore aligns the mean values of these small fragments to the reference. After alignment, we concatenate all fragments that map to the same k-mer into a larger segment, analogous to the "eventalign" output in Nanopolish. For RNA modification estimation, we use only the mean signal value of each reconstructed event."

To avoid ambiguity, we have also revised the sentence on page 5 to more clearly distinguish the roles of Nanopolish and SegPore in the workflow. The updated sentence now reads:

“Nanopolish provides two independent commands: “polya” and “eventalign”. The “polya” command identifies the adapter, poly(A) tail, and transcript region in the raw signal, which we refer to as the poly(A) detection results. The raw signal segment corresponding to the poly(A) tail is used to standardize the raw signal for each read. The “eventalign” command aligns the raw signal to a reference sequence, assigning a signal segment to individual k-mers in the reference. It also computes summary statistics (e.g., mean, standard deviation) from the signal segment for each k-mer. Each k-mer together with its corresponding signal features is termed an event. These event features are then passed into downstream tools such as m6Anet and CHEUI for RNA modification detection. For full transcriptome analysis (Figure 3), we extract the aligned raw signal segment and reference sequence segment from Nanopolish’s events for each read by using the first and last events as start and end points. For in vitro transcription (IVT) data with a known reference sequence (Figure 4), we extract the raw signal segment corresponding to the transcript region for each input read based on Nanopolish’s poly(A) detection results.”

(6) Page 5, “Since the polyA tail provides a stable reference, we normalize the raw current signals across reads, ensuring that the mean and standard deviation of the polyA tail are consistent across all reads.”

Perhaps I misread this statement: I interpret it as using the PolyA tail to do the normalization, rather than using the rest of the signal to do the normalization, and that results in consistent PolyA tails across all reads.

If it's the latter, this should be clarified, and a little detail on how the normalization is done should be added, but if my first interpretation is correct:

I'm not sure if its standard deviation is consistent across reads. The (true) value spread in this section of a read should be fairly limited compared to the rest of the signal in the read, so the noise would influence the scale quite quickly, and such noise might be introduced to pores wearing down and other technical influences. Is this really better than using the non-PolyA tail part of the reads signal, using Median Absolute Deviation to scale for a first alignment round, then re-fitting the signal scaling using Theil Sen on the resulting alignments (assigned read signal vs reference expected signal), as Tombo/Nanopolish (can) do?

Additionally, this kind of normalization should have been part of the Nanopolish eventalign already, can this not be re-used? If it's done differently it may result in different distributions than the ONT kmer table obtained for the next step.

Thank you for this detailed and thoughtful comment. We apologize for the confusion. The poly(A) tail-based normalization is indeed explained in Supplementary Note 1, Section 3, but we agree that the motivation needed to be clarified in the main text.

We have now added the following sentence in the revised manuscript (before the original statement on page 5 to provide clearer context:

“Due to inherent variability between nanopores in the sequencing device, the baseline levels and standard deviations of k-mer signals can differ across reads, even for the same transcript. To standardize the signal for downstream analyses, we extract the raw current signal segments corresponding to the poly(A) tail of each read. Since the poly(A) tail provides a stable reference, we normalize the raw current signals across reads, ensuring that the mean and standard deviation of the poly(A) tail are consistent across all reads. This step is crucial for reducing.....”

We chose to use the poly(A) tail for normalization because it is sequence-invariant—i.e., all poly(A) tails consist of identical k-mers, unlike transcript sequences which vary in composition. In contrast, using the transcript region for normalization can introduce biases: for instance, reads with more diverse k-mers (having inherently broader signal distributions) would be forced to match the variance of reads with more uniform k-mers, potentially distorting the baseline across k-mers.

In our newly added RNA004 benchmark experiment, we used the default normalization provided by f5c, which does not include poly(A) tail normalization. Despite this, SegPore was still able to mask out noise and outperform both f5c and Uncalled4, demonstrating that our segmentation method is robust to different normalization strategies.

(7) Page 7, “The initialization of the 5mer parameter table is a critical step in SegPore’s workflow. By leveraging ONT’s established kmer models, we ensure that the initial estimates for unmodified 5mers are grounded in empirical data.”

It looks like the method uses Nanopolish for a first alignment, then improves the segmentation matching the reference sequence/expected 5mer values. I thought the Nanopolish model/tables are based on the same data, or similarly obtained. If they are different, then why the switch of kmer model? Now the original alignment may have been based on other values, and thus the alignment may seem off with the expected kmer values of this table.

Thank you for this insightful question. To clarify, SegPore uses Nanopolish only to identify the poly(A) tail and transcript regions from the raw signal. In the bulk in vivo data analysis, we use Nanopolish’s first event as the start and the last event as the end to extract the aligned raw signal chunk and its corresponding reference sequence. Since SegPore relies on Nanopolish solely to delineate the transcript region for each read, it independently aligns the raw signals to the reference sequence without refining or adjusting Nanopolish’s segmentation results.

While SegPore’s 5-mer parameter table is initially seeded using ONT’s published unmodified k-mer models, we acknowledge that empirical signal values may deviate from these reference models due to run-specific technical variation and the presence of RNA modifications. For this reason, SegPore includes a parameter re-estimation step to refine the mean and standard deviation values of each k-mer based on the current dataset.

The re-estimation process consists of two layers. In the outer layer, we select a set of 5mers that exhibit both modified and unmodified states based on the GMM results (Section 6 of Supplementary Note 1), while the remaining 5mers are assumed to have only unmodified states. In the inner layer, we align the raw signals to the reference sequences using the 5mer parameter table estimated in the outer layer (Section 5 of Supplementary Note 1). Based on the alignment results, we update the 5mer parameter table in the outer layer. This two-layer process is generally repeated for 3~5 iterations until the 5mer parameter table converges. This re-estimation ensures that:

- (1) The adjusted 5mer signal baselines remain close to the ONT reference (for consistency);
- (2) The alignment score between the observed signal and the reference sequence is optimized (as detailed in Equation 11, Section 5 of Supplementary Note 1);
- (3) Only 5mers that show a clear difference between the modified and unmodified components in the GMM are considered subject to modification.

By doing so, SegPore achieves more accurate signal alignment independent of Nanopolish’s models, and the alignment is directly tuned to the data under analysis.

(8) Page 9, “The output of the alignment algorithm is an eventalign, which pairs the base blocks with the 5mers from the reference sequence for each read (Fig. 1C).”

“Modification prediction

After obtaining the eventalign results, we estimate the modification state of each motif using the 5mer parameter table.”

This wording seems to have been introduced on page 5 but (also there) reads a bit confusingly as the name of the output format, file, and function are now named the exact same “eventalign”. I assume the obtained eventalign results now refer to the output of your HHMM, and not the original Nanopolish eventalign results, based on context only, but I'd rather have a clear naming that enables more differentiation.

We apologize for the confusion. We have revised the sentence as follows for clarity:

“A detailed description of both alignment algorithms is provided in Supplementary Note 1. The output of the alignment algorithm is an alignment that pairs the base blocks with the 5mers from the reference sequence for each read (Fig. 1C). Base blocks aligned to the same 5-mer are concatenated into a single raw signal segment (referred to as an “event”), from which various features—such as start and end positions, mean current, and standard deviation—are extracted. Detailed derivation of the mean and standard deviation is provided in Section 5.3 in Supplementary Note 1. In the remainder of this paper, we refer to these resulting events as the output of eventalign analysis or the segmentation task.”

(9) Page 9, “Since a single 5mer can be aligned with multiple base blocks, we merge all aligned base blocks by calculating a weighted mean. This weighted mean represents the single base block mean aligned with the given 5mer, allowing us to estimate the modification state for each site of a read.”

I assume the weights depend on the length of the segment but I don't think it is explicitly stated while it should be.

Thank you for the helpful observation. To improve clarity, we have moved this explanation to the last paragraph of the previous section (see response to point 8), where we describe the segmentation process in more detail.

Additionally, a complete explanation of how the weighted mean is computed is provided in Section 5.3 of Supplementary Note 1. It is derived from signal points that are assigned to a given 5mer.

(10) Page 10, “Afterward, we manually adjust the 5mer parameter table using heuristics to ensure that the modified 5mer distribution is significantly distinct from the unmodified distribution.”

Using what heuristics? If this is explained in the supplementary notes then please refer to the exact section.

Thank you for pointing this out. The heuristics used to manually adjust the 5mer parameter table are indeed explained in detail in Section 7 of Supplementary Note 1.

To clarify this in the manuscript, we have revised the sentence as follows:

“Afterward, we manually adjust the 5mer parameter table using heuristics to ensure that the modified 5mer distribution is significantly distinct from the unmodified distribution (see details in Section 7 of Supplementary Note 1).”

(11) Page 10, “Once the table is fixed, it is used for RNA modification estimation in the test data without further updates.”

By what tool/algorithm? Perhaps it is your own implementation, but with the next section going into segmentation benchmarking and using Nanopolish before this seems undefined.

Thank you for pointing this out. We use our own implementation. See Algorithm 3 in Section 6 of Supplementary Note 1.

We have revised the sentence for clarity:

“Once a stabilized 5mer parameter table is estimated from the training data, it is used for RNA modification estimation in the test data without further updates. A more detailed description of the GMM re-estimation process is provided in Section 6 of Supplementary Note 1.”

(12) Page 11, “A 5mer was considered significantly modified if its read coverage exceeded 1,500 and the distance between the means of the two Gaussian components in the GMM was greater than 5.”

Considering the scaling done before also not being very detailed in what range to expect, this cutoff doesn't provide any useful information. Is this a pA value?

Thank you for the observation. Yes, the value refers to the current difference measured in picoamperes (pA). To clarify this, we have revised the sentence in the manuscript to include the unit explicitly:

“A 5mer was considered significantly modified if its read coverage exceeded 1,500 and the distance between the means of the two Gaussian components in the GMM was greater than 5 picoamperes (pA).”

(13) Page 13, “The raw current signals, as shown in Figure 1B.”

Wrong figure? Figure 2B seems logical.

Thank you for catching this. You are correct—the reference should be to Figure 2B, not Figure 1B. We have corrected this in the revised manuscript.

(14) Page 14, Figure 2A, these figures supposedly support the jiggle hypothesis but the examples seem to match only half the explanation. Any of these jiggles seem to be followed shortly by another in the opposite direction, and the amplitude seems to match better within each such pair than the next or previous segments. Perhaps there is a better explanation still, and this behaviour can be modelled as such instead.

Thank you for your comment. We acknowledge that the observed signal patterns may appear ambiguous and could potentially suggest alternative explanations. However, as shown in Figure 2A, the red dots tend to align closely with the baseline of the previous state, while the blue dots align more closely with the baseline of the next state. We interpret this as evidence for the “jiggling” hypothesis, where k-mer temporarily oscillates between adjacent states during translocation.

That said, we agree that more sophisticated models could be explored to better capture this behavior, and we welcome suggestions or references to alternative models. We will consider this direction in future work.

(15) Page 15, “This occurs because subtle transitions within a base block may be mistaken for transitions between blocks, leading to inflated transition counts.”

Is it really a "subtle transition" if it happens within a base block? It seems this is not a transition and thus shouldn't be named as such.

Thank you for pointing this out. We agree that the term “subtle transition” may be misleading in this context. We revised the sentence to clarify the potential underlying cause of the inflated transition counts:

“This may be due to a base block actually corresponding to a sub-state of a single 5mer, rather than each base block corresponding to a full 5mer, leading to inflated transition counts. To address this issue, SegPore’s alignment algorithm was refined to merge multiple base blocks (which may represent sub-states of the same 5mer) into a single 5mer, thereby facilitating further analysis.”

(16) Page 15, “The SegPore “eventalign” output is similar to Nanopolish’s “eventalign” command.”

To the output of that command, I presume, not to the command itself.

Thank you for pointing out the ambiguity. We have revised the sentence for clarity:

“The final outputs of SegPore are the events and modification state predictions. SegPore’s events are similar to the outputs of Nanopolish’s “eventalign” command, in that they pair raw current signal segments with the corresponding RNA reference 5-mers. Each 5-mer is associated with various features — such as start and end positions, mean current, and standard deviation — derived from the paired signal segment.”

(17) Page 15, “For selected 5mers, SegPore also provides the modification rate for each site and the modification state of that site on individual reads.”

What selection? Just all kmers with a possible modified base or a more specific subset?

We revised the sentence to clarify the selection criteria:

“For selected 5mers that exhibit both a clearly unmodified and a clearly modified signal component, SegPore reports the modification rate at each site, as well as the modification state of that site on individual reads.”

(18) Page 16, “A key component of SegPore is the 5mer parameter table, which specifies the mean and standard deviation for each 5mer in both modified and unmodified states (Figure 2A).”

Wrong figure?

Thank you for pointing this out. You are correct—it should be Figure 1A, not Figure 2A. We intended to visually illustrate the structure of the 5mer parameter table in Figure 1A, and we have corrected this reference in the revised manuscript.

(19) Page 16, Table 1, I can't quite tell but I assume this is based on all kmers in the table, not just a m6A modified subset. A short added statement to make this clearer would help.

Yes, you are right—it is averaged over all 5mers. We have revised the sentence for clarity as follows:

" As shown in Table 1, SegPore consistently achieved the best performance averaged on all 5mers across all datasets....."

(20) Page 16, "Since the peaks (representing modified and unmodified states) are separable for only a subset of 5mers, SegPore can provide modification parameters for these specific 5mers. For other 5mers, modification state predictions are unavailable."

Can this be improved using some heuristics rather than the 'distance of 5' cutoff as described before? How small or big is this subset, compared to how many there should be to cover all cases?

We agree that more sophisticated strategies could potentially improve performance. In this study, we adopted a relatively conservative approach to minimize false positives by using a heuristic cutoff of 5 picoamperes. This value was selected empirically and we did not explore alternative cutoffs. Future work could investigate more refined or data-driven thresholding strategies.

(21) Page 16, "Tombo used the "resquiggle" method to segment the raw signals, and we standardized the segments using the polyA tail to ensure a fair comparison."

I don't know what or how something is "standardized" here.

Standardized' refers to the poly(A) tail-based signal normalization described in our response to point 6. We applied this normalization to Tombo's output to ensure a fair comparison across methods. Without this standardization, Tombo's performance was notably worse. We revised the sentence as follows:

"Tombo used the "resquiggle" method to segment the raw signals, and we standardized the segments using the poly(A) tail to ensure a fair comparison (See preprocessing section in Materials and Methods)."

(22) Page 16, "To benchmark segmentation performance, we used two key metrics: (1) the log-likelihood of the segment mean, which measures how closely the segment matches ONT's 5mer parameter table (used as ground truth), and (2) the standard deviation (std) of the segment, where a lower std indicates reduced noise and better segmentation quality. If the raw signal segment aligns correctly with the corresponding 5mer, its mean should closely match ONT's reference, yielding a high log-likelihood. A lower std of the segment reflects less noise and better performance overall."

Here the segmentation part becomes a bit odd:

A: Low std can be/is achieved by dropping any noisy bits, making segments really small (partly what happens here with the transition segments). This may be 'true' here, in the sense that the transition is not really part of the segment, but the comparison table is a bit meaningless as the other tools forcibly assign all data to kmers, instead of ignoring parts as transition states. In other words, it is a benchmark that is easy to cheat by assigning more data to noise/transition states.

B: The values shown are influenced by the alignment made between the read and expected reference signal. Especially Tombo tends to forcibly assign data to whatever looks the most similar nearby rather than providing the correct alignment. So the "benchmark of the segmentation performance" is more of an "overall benchmark of the raw signal alignment". Which is still a good, useful thing, but the text seems to suggest something else.

Thank you for raising these important concerns regarding the segmentation benchmarking.

Regarding point A, the base blocks aligned to the same 5mer are concatenated into a single segment, including the short transition blocks between them. These transition blocks are typically very short (4~10 signal points, average 6 points), while a typical 5mer segment contains around 20~60 signal points. To assess whether SegPore's performance is inflated by excluding transition segments, we conducted an additional comparison: we removed 6 boundary signal points (3 from the start and 3 from the end) from each 5mer segment in Nanopolish and Tombo's results to reduce potential noise. The new comparison table is shown in the following:

SegPore consistently demonstrates superior performance. Its key contribution lies in its ability to recognize structured noise in the raw signal and to derive more accurate mean and standard deviation values that more faithfully represent the true state of the k-mer in the pore. The improved mean estimates are evidenced by the clearly separated peaks of modified and unmodified 5mers in Figures 3A and 4B, while the improved standard deviation is reflected in the segmentation benchmark experiments.

Regarding point B, we apologize for the confusion. We have added a new paragraph to the introduction to clarify that the segmentation task indeed includes the alignment step.

“The general workflow of Nanopore direct RNA sequencing (DRS) data analysis is as follows. First, the raw electrical signal from a read is basecalled using tools such as Guppy or Dorado, which produce the nucleotide sequence of the RNA molecule. However, these basecalled sequences do not include the precise start and end positions of each ribonucleotide (or k-mer) in the signal. Because basecalling errors are common, the sequences are typically mapped to a reference genome or transcriptome using minimap2 to recover the correct reference sequence. Next, tools such as Nanopolish and Tombo align the raw signal to the reference sequence to determine which portion of the signal corresponds to each k-mer. We define this process as the segmentation task, referred to as "eventalign" in Nanopolish. Based on this alignment, Nanopolish extracts various features—such as the start and end positions, mean, and standard deviation of the signal segment corresponding to a k-mer. This signal segment or its derived features is referred to as an "event" in Nanopolish. The resulting events serve as input for downstream RNA modification detection tools such as m6Anet and CHEUI.”

(23) Page 17 “Given the comparable methods and input data requirements, we benchmarked SegPore against several baseline tools, including Tombo, MINES (26), Nanom6A (27), m6Anet, Epinano (28), and CHEUI (29).”

It seems m6Anet is actually Nanopolish+m6Anet in Figure 3C, this needs a minor clarification here.

m6Anet uses Nanopolish's estimated events as input by default.

(24) Page 18, Figure 3, A and B are figures without any indication of what is on the axis and from the text I believe the position next to each other on the x-axis rather than overlapping is meaningless, while their spread is relevant, as we're looking at the distribution of raw values for this 5mer. The figure as is is rather confusing.

Thanks for pointing out the confusion. We have added concrete values to the axes in Figures 3A and 3B and revised the figure legend as follows in the manuscript:

“(A) Histogram of the estimated mean from current signals mapped to an example m6A-modified genomic location (chr10:128548315, GGACT) across all reads in the training data,

comparing Nanopolish (left) and SegPore (right). The x-axis represents current in picoamperes (pA).

(B) Histogram of the estimated mean from current signals mapped to the GGACT motif at all annotated m6A-modified genomic locations in the training data, again comparing Nanopolish (left) and SegPore (right). The x-axis represents current in picoamperes (pA)."

(25) Page 18 "SegPore's results show a more pronounced bimodal distribution in the raw signal segment mean, indicating clearer separation of modified and unmodified signals."

Without knowing the correct values around the target kmer (like Figure 4B), just the more defined bimodal distribution could also indicate the (wrongful) assignment of neighbouring kmer values to this kmer instead, hence this statement lacks some needed support, this is just one interpretation of the possible reasons.

Thank you for the comment. We have added concrete values to Figures 3A and 3B to support this point. Both peaks fall within a reasonable range: the unmodified peak (125 pA) is approximately 1.17 pA away from its reference value of 123.83 pA, and the modified peak (118 pA) is around 7 pA away from the unmodified peak. This shift is consistent with expected signal changes due to RNA modifications (usually less than 10 pA), and the magnitude of the difference suggests that the observed bimodality is more likely caused by true modification events rather than misalignment.

(26) Page 18 "Furthermore, when pooling all reads mapped to m6A-modified locations at the GGACT motif, SegPore showed prominent peaks (Fig. 3B), suggesting reduced noise and improved modification detection."

I don't think the prominent peaks directly suggest improved detection, this statement is a tad overreaching.

We revised the sentence to the following:

"SegPore exhibited more distinct peaks (Fig. 3B), indicating reduced noise and potentially enabling more reliable modification detection".

(27) Page18 "(2) direct m6A predictions from SegPore's Gaussian Mixture Model (GMM), which is limited to the six selected 5mers."

The 'six selected' refers to what exactly? Also, 'why' this is limited to them is also unclear as it is, and it probably would become clearer if it is clearly defined what this refers to.

It is explained the page 16 in the SegPore's workflow in the original manuscript as follows:

"A key component of SegPore is the 5mer parameter table, which specifies the mean and standard deviation for each 5mer in both modified and unmodified states (Fig. 2A1A). Since the peaks (representing modified and unmodified states) are separable for only a subset of 5mers, SegPore can provide modification parameters for these specific 5mers. For other 5mers, modification state predictions are unavailable."

e select a small set of 5mers that show clear peaks (modified and unmodified 5mers) in GMM in the m6A site-level data analysis. These 5mers are provided in Supplementary Fig. S2C, as explained in the section "m6A site level benchmark" in the Material and Methods (page 12 in the original manuscript).

"...transcript locations into genomic coordinates. It is important to note that the 5mer parameter table was not re-estimated for the test data. Instead, modification states for each read were directly estimated using the fixed 5mer parameter table. Due to the differences

between human (Supplementary Fig. S2A) and mouse (Supplementary Fig. S2B), only six 5mers were found to have m6A annotations in the test data's ground truth (Supplementary Fig. S2C). For a genomic location to be identified as a true m6A modification site, it had to correspond to one of these six common 5mers and have a read coverage of greater than 20. SegPore derived the ROC and PR curves for benchmarking based on the modification rate at each genomic location....”

We have updated the sentence as follows to increase clarity:

“which is limited to the six selected 5mers that exhibit clearly separable modified and unmodified components in the GMM (see Materials and Methods for details).”

(28) Page 19, Figure 4C, the blue 'Unmapped' needs further explanation. If this means the segmentation+alignment resulted in simply not assigning any segment to a kmer, this would indicate issues in the resulting mapping between raw data and kmers as the data that probably belonged to this kmer is likely mapped to a neighbouring kmer, possibly introducing a bimodal distribution there.

This is due to deletion event in the full alignment algorithm. See Page 8 of SupplementaryNote1:

During the traceback step of the dynamic programming matrix, not every 5mer in the reference sequence is assigned a corresponding raw signal fragment—particularly when the signal's mean deviates substantially from the expected mean of that 5mer. In such cases, the algorithm considers the segment to be generated by an unknown 5mer, and the corresponding reference 5mer is marked as unmapped.

(29) Page 19, “For six selected m6A motifs, SegPore achieved an ROC AUC of 82.7% and a PR AUC of 38.7%, earning the third-best performance compared with deep leaning methods m6Anet and CHEUI (Fig. 3D).”

How was this selection of motifs made, are these related to the six 5mers in the middle of Supplementary Figure S2? Are these the same six as on page 18? This is not clear to me.

It is the same, see the response to point 27.

(30) Page 21 “Biclustering reveals that modifications at the 6th, 7th, and 8th genomic locations are specific to certain clusters of reads (clusters 4, 5, and 6), while the first five genomic locations show similar modification patterns across all reads.”

This reads rather confusingly. Both the '6th, 7th, and 8th genomic locations' and 'clusters 4,5,6' should be referred to in clearer terms. Either mark them in the figure as such or name them in the text by something that directly matches the text in the figure.

We have added labels to the clusters and genomic locations Figure 4C, and revised the sentence as follows:

“Biclustering reveals that modifications at g6 are specific to cluster C4, g7 to cluster C5, and g8 to cluster C6, while the first five genomic locations (g1 to g5) show similar modification patterns across all reads.”

(31) Page 21, “We developed a segmentation algorithm that leverages the jiggling property in the physical process of DRS, resulting in cleaner current signals for m6A identification at both the site and single-molecule levels.”

Leverages, or just 'takes into account'?

We designed our HHMM specifically based on the jiggling hypothesis, so we believe that using the term “leverage” is appropriate.

(32) Page 21, “Our results show that m6Anet achieves superior performance, driven by SegPore’s enhanced segmentation.”

Superior in what way? It barely improves over Nanopolish in Figure 3C and is outperformed by other methods in Figure 3D. The segmentation may have improved but this statement says something is 'superior' driven by that 'enhanced segmentation', so that cannot refer to the segmentation itself.

We revise it as follows in the revised manuscript:

“Our results demonstrate that SegPore’s segmentation enables clear differentiation between m6A-modified and unmodified adenosines.”

(33) Page 21, “In SegPore, we assume a drastic change between two consecutive 5mers, which may hold for 5mers with large difference in their current baselines but may not hold for those with small difference.”

The implications of this assumption don't seem highlighted enough in the work itself and may be cause for falsely discovering bi-modal distributions. What happens if such a 5mer isn't properly split, is there no recovery algorithm later on to resolve these cases?

We agree that there is a risk of misalignment, which can result in a falsely observed bimodal distribution. This is a known and largely unavoidable issue across all methods, including deep neural network-based methods. For example, many of these models rely on a CTC (Connectionist Temporal Classification) layer, which implicitly performs alignment and may also suffer from similar issues.

Misalignment is more likely when the current baselines of neighboring k-mers are close. In such cases, the model may struggle to confidently distinguish between adjacent k-mers, increasing the chance that signals from neighboring k-mers are incorrectly assigned. Accurate baseline estimation for each k-mer is therefore critical—when baselines are accurate, the correct alignment typically corresponds to the maximum likelihood.

We have added the following sentence to the discussion to acknowledge this limitation:

“As with other RNA modification estimation methods, SegPore can be affected by misalignment errors, particularly when the baseline signals of adjacent k-mers are similar. These cases may lead to spurious bimodal signal distributions and require careful interpretation.”

(34) Page 21, “Currently, SegPore models only the modification state of the central nucleotide within the 5mer. However, modifications at other positions may also affect the signal, as shown in Figure 4B. Therefore, introducing multiple states to the 5mer could help to improve the performance of the model.”

The meaning of this statement is unclear to me. Is SegPore unable to combine the information of overlapping kmers around a possibly modified base (central nucleotide), or is this referring to having multiple possible modifications in a single kmer (multiple states)?

We mean there can be modifications at multiple positions of a single 5mer, e.g. C m5C m6A m7G T. We have revised the sentence to:

“Therefore, introducing multiple states for a 5mer to account for modifications at multiple positions within the same 5mer could help to improve the performance of the model.”

(35) Page 22, “This causes a problem when apply DNN-based methods to new dataset without short read sequencing-based ground truth. Human could not confidently judge if a predicted m6A modification is a real m6A modification.”

Grammatical errors in both these sentences. For the 'Human could not' part, is this referring to a single person's attempt or more extensively tested?

Thanks for the comment. We have revised the sentence as follows:

“This poses a challenge when applying DNN-based methods to new datasets without short-read sequencing-based ground truth. In such cases, it is difficult for researchers to confidently determine whether a predicted m6A modification is genuine (see Supplementary Figure S5).”

(36) Page 22, “...which is easier for human to interpret if a predicted m6A site is real.”

"a" human, but also this probably meant to say 'whether' instead of 'if', or 'makes it easier'.

Thanks for the advice. We have revise the sentence as follows:

“One can generally observe a clear difference in the intensity levels between 5mers with an m6A and those with a normal adenosine, which makes it easier for a researcher to interpret whether a predicted m6A site is genuine.”

(37) Page 22, “...and noise reduction through its GMM-based approach...”

Is the GMM providing noise reduction or segmentation?

Yes, we agree that it is not relevant. We have removed the sentence in the revised manuscript as follows:

“Although SegPore provides clear interpretability and noise reduction through its GMM-based approach, there is potential to explore DNN-based models that can directly leverage SegPore's segmentation results.”

(38) Page 23, “SegPore effectively reduces noise in the raw signal, leading to improved m6A identification at both site and single-molecule levels...”

Without further explanation in what sense this is meant, 'reduces noise' seems to overreach the abilities, and looks more like 'masking out'.

Following the reviewer's suggestion, we change it to 'mask out' in the revised manuscript.

“SegPore effectively masks out noise in the raw signal, leading to improved m6A identification at both site and single-molecule levels.”

Reviewer #3 (Recommendations for the authors):

I recommend the publication of this manuscript, provided that the following comments (and the comments above) are addressed.

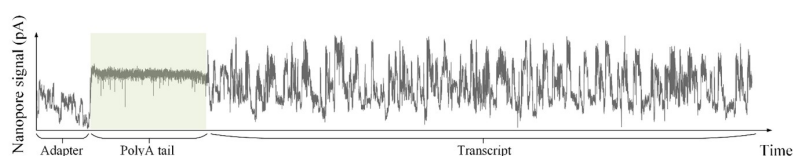
In general, the authors state that SegPore represents an improvement on existing software. These statements are largely unquantified, which erodes their credibility. I have specified several of these in the Minor comments section.

Page 5, Preprocessing: The authors comment that the poly(A) tail provides a stable reference that is crucial for the normalisation of all reads. How would this step handle reads that have variable poly(A) tail lengths? Or have interrupted poly(A) tails (e.g. in the case of mRNA vaccines that employ a linker sequence)?

We apologize for the confusion. The poly(A) tail-based normalization is explained in Supplementary Note 1, Section 3.

As shown in Author response image 1 below, the poly(A) tail produces a characteristic signal pattern—a relatively flat, squiggly horizontal line. Due to variability between nanopores, raw current signals often exhibit baseline shifts and scaling of standard deviations. This means that the signal may be shifted up or down along the y-axis and stretched or compressed in scale.

Author response image 1.



The normalization remains robust with variable poly(A) tail lengths, as long as the poly(A) region is sufficiently long. The linker sequence will be assigned to the adapter part rather than the poly(A) part.

To improve clarity in the revised manuscript, we have added the following explanation:

“Due to inherent variability between nanopores in the sequencing device, the baseline levels and standard deviations of k-mer signals can differ across reads, even for the same transcript. To standardize the signal for downstream analyses, we extract the raw current signal segments corresponding to the poly(A) tail of each read. Since the poly(A) tail provides a stable reference, we normalize the raw current signals across reads, ensuring that the mean and standard deviation of the poly(A) tail are consistent across all reads. This step is crucial for reducing.....”

We chose to use the poly(A) tail for normalization because it is sequence-invariant—i.e., all poly(A) tails consist of identical k-mers, unlike transcript sequences which vary in composition. In contrast, using the transcript region for normalization can introduce biases: for instance, reads with more diverse k-mers (having inherently broader signal distributions) would be forced to match the variance of reads with more uniform k-mers, potentially distorting the baseline across k-mers.

Page 7, 5mer parameter table: r9.4. 180mv_70bps_5mer_RNA is an older kmer model (>2 years). How does your method perform with the newer RNA kmer models that do permit the detection of multiple ribonucleotide modifications? Addressing this comment is crucial because it is feasible that SegPore will underperform in comparison to the newer RNA base caller models (requiring the use of RNA004 datasets).

Thank you for highlighting this important point. For RNA004, we have updated SegPore to ensure compatibility with the latest kit. In our revised manuscript, we demonstrate that the translocation-based segmentation hypothesis remains valid for RNA004, as supported by new analyses presented in the supplementary Figure S4.

Additionally, we performed a new benchmark with f5c and Uncalled4 in RNA004 data in the revised manuscript (Table 2), where SegPore exhibit a better performance than f5c and Uncalled4.

We agree that benchmarking against the latest Dorado models—specifically rna004_130bps_hac@v5.1.0 and rna004_130bps_sup@v5.1.0, which include built-in modification detection capabilities—would provide valuable context for evaluating the utility of SegPore. However, generating a comprehensive k-mer parameter table for RNA004 requires a large, well-characterized dataset. At present, such data are limited in the public domain. Additionally, Dorado is developed by ONT and its internal training data have not been released, making direct comparisons difficult.

Our current focus is on improving raw signal segmentation quality, which are upstream tasks critical to many downstream analyses, including RNA modification detection. Future work may include benchmarking SegPore against models like Dorado once appropriate data become available.

The Methods and Results sections contain redundant information - please streamline the information in these sections and reduce the redundancy. For example, the benchmarking section may be better situated in the Results section.

Following your advice, we have removed redundant texts about the Segmentation benchmark from Materials and Methods in the revised manuscript.

Minor comments

(1) Introduction

Page 3: "By incorporating these dynamics into its segmentation algorithm...". Please provide an example of how motor protein dynamics can impact RNA translocation. In particular, please elaborate on why motor protein dynamics would impact the translocation of modified ribonucleotides differently to canonical ribonucleotides. This is provided in the results, but please also include details in the Introduction.

Following your advice, we added one sentence to explain how the motor protein affect the translocation of the DNA/RNA molecule in the revised manuscript.

"This observation is also supported by previous reports, in which the helicase (the motor protein) translocates the DNA strand through the nanopore in a back-and-forth manner. Depending on ATP or ADP binding, the motor protein may translocate the DNA/RNA forward or backward by 0.5-1 nucleotides."

As far as we understand, this translocation mechanism is not specific to modified or unmodified nucleotides. For further details, we refer the reviewer to the original studies cited.

Page 3: "This lack of interpretability can be problematic when applying these methods to new datasets, as researchers may struggle to trust the predictions without a clear understanding of how the results were generated." Please provide details and citations as to why researchers would struggle to trust the predictions of m6Anet. Is it due to a

lack of understanding of how the method works, or an empirically demonstrated lack of reliability?

Thank you for pointing this out. The lack of interpretability in deep learning models such as m6Anet stems primarily from their “black-box” nature—they provide binary predictions (modified or unmodified) without offering clear reasoning or evidence for each call.

When we examined the corresponding raw signals, we found it difficult to visually distinguish whether a signal segment originated from a modified or unmodified ribonucleotide. The difference is often too subtle to be judged reliably by a human observer. This is illustrated in the newly added Supplementary Figure S5, which shows Nanopolish-aligned raw signals for the central 5mer GGACT in Figure 4B, displayed both uncolored and colored by modification state (according to the ground truth).

Although deep neural networks can learn subtle, high-dimensional patterns in the signal that may not be readily interpretable, this opacity makes it difficult for researchers to trust the predictions—especially in new datasets where no ground truth is available. The issue is not necessarily an empirically demonstrated lack of reliability, but rather a lack of transparency and interpretability.

We have updated the manuscript accordingly and included Supplementary Figure S5 to illustrate the difficulty in interpreting signal differences between modified and unmodified states.

Page 3: "Instead of relying on complex, opaque features...". Please provide evidence that the research community finds the figures generated by m6Anet to be difficult to interpret, or delete the sections relating to its perceived lack of usability.

See the figure provided in the response to the previous point. We added a reference to this figure in the revised manuscript.

“Instead of relying on complex, opaque features (see Supplementary Figure S5), SegPore leverages baseline current levels to distinguish between.....”

(2) Materials and Methods

Page 5, Preprocessing: "We begin by performing basecalling on the input fast5 file using Guppy, which converts the raw signal data into base sequences.". Please change "base" to ribonucleotide.

Revised as requested.

Page 5 and throughout, please refer to poly(A) tail, rather than polyA tail throughout.

Revised as requested.

Page 5, Signal segmentation via hierarchical Hidden Markov model: "...providing more precise estimates of the mean and variance for each base block, which are crucial for downstream analyses such as RNA modification prediction." Please specify which method your HHMM method improves upon.

Thank you for the suggestion. Since this section does not include a direct comparison, we revised the sentence to avoid unsupported claims. The updated sentence now reads:

"...providing more precise estimates of the mean and variance for each base block, which are crucial for downstream analyses such as RNA modification prediction."

Page 10, GMM for 5mer parameter table re-estimation: "Typically, the process is repeated three to five times until the 5mer parameter table stabilizes." How is the stabilisation of the 5mer parameter table quantified? What is a reasonable cut-off that would demonstrate adequate stabilisation of the 5mer parameter table?

Thank you for the comment. We assess the stabilization of the 5mer parameter table by monitoring the change in baseline values across iterations. If the absolute change in baseline values for all 5mers is less than $1e-5$ between two consecutive iterations, we consider the estimation to have stabilized.

Page 11, M6A site level benchmark: why were these datasets selected? Specifically, why compare human and mouse ribonucleotide modification profiles? Please provide a justification and a brief description of the experiments that these data were derived from, and why they are appropriate for benchmarking SegPore.

Thank you for the comment. These data are taken from a previous benchmark study about m6A estimation from RNA002 data in the literature (<https://doi.org/10.1038/s41467-023-37596-5>). We think the data are appropriate here.

Thank you for the comment. The datasets used were taken from a previous benchmark study on m6A estimation using RNA002 data (<https://doi.org/10.1038/s41467-023-37596-5>). These datasets include human and mouse transcriptomes and have been widely used to evaluate the performance of RNA modification detection tools. We selected them because (i) they are based on RNA002 chemistry, which matches the primary focus of our study, and (ii) they provide a well-characterized and consistent benchmark for assessing m6A detection performance. Therefore, we believe they are appropriate for validating SegPore.

(3) Results

Page 13, RNA translocation hypothesis: "The raw current signals, as shown in Fig. 1B...". Please check/correct figure reference - Figure 1B does not show raw current signals.

Thank you for pointing this out. The correct reference should be Figure 2B. We have updated the figure citation accordingly in the revised manuscript.

Page 19, m6A identification at the site level: "For six selected m6A motifs, SegPore achieved an ROC AUC of 82.7% and a PR AUC of 38.7%, earning the third best performance compared with deep learning methods m6Anet and CHEUI (Fig. 3D)." SegPore performs third best of all deep learning methods. Do the authors recommend its use in conjunction with m6Anet for m6A detection? Please clarify in the text.

This sentence aims to convey that SegPore alone can already achieve good performance. If interpretability is the primary goal, we recommend using SegPore on its own. However, if the objective is to identify more potential m6A sites, we suggest using the combined approach of SegPore and m6Anet. That said, we have chosen not to make explicit recommendations in the main text to avoid oversimplifying the decision or potentially misleading readers.

Page 19, m6A identification at the single molecule level: "one transcribed with m6A and the other with normal adenosine". I assume that this should be adenine? Please replace adenosine with adenine throughout.

Thank you for pointing this out. We have revised the sentence to use "adenine" where appropriate. In other instances, we retain "adenosine" when referring specifically to adenine bound to a ribose sugar, which we believe is suitable in those contexts.

Page 19, m6A identification at the single molecule level: "We used 60% of the data for training and 40% for testing". How many reads were used for training and how many for testing? Please comment on why these are appropriate sizes for training and testing datasets.

In total, there are 1.9 million reads, with 1.14 million used for training and 0.76 million for testing (60% and 40%, respectively). We chose this split to ensure that the training set is sufficiently large to reliably estimate model parameters, while the test set remains substantial enough to robustly evaluate model performance. Although the ratio was selected somewhat arbitrarily, it balances the need for effective training with rigorous validation.

(4) Discussion

Page 21: "We believe that the de-noised current signals will be beneficial for other downstream tasks." Which tasks? Please list an example.

We have revised the text for clarity as follows:

"We believe that the de-noised current signals will be beneficial for other downstream tasks, such as the estimation of m5C, pseudouridine, and other RNA modifications."

Page 22: "One can generally observe a clear difference in the intensity levels between 5mers with a m6A and normal adenosine, which is easier for human to interpret if a predicted m6A site is real." This statement is vague and requires qualification. Please reference a study that demonstrates the human ability to interpret two similar graphs, and demonstrate how it relates to the differences observed in your data.

We apologize for the confusion. We have revised the sentence as follows:

"One can generally observe a clear difference in the intensity levels between 5mers with an m6A and those with a normal adenosine, which makes it easier for a researcher to interpret whether a predicted m6A site is genuine."

We believe that Figures 3A, 3B, and 4B effectively illustrate this concept.

Page 23: How long does SegPore take for its analyses compared to other similar tools? How long would it take to analyse a typical dataset?

We have added run-time statistics for datasets of varying sizes in the revised manuscript (see Supplementary Figure S6). This figure illustrates SegPore's performance across different data volumes to help estimate typical processing times.

(5) Figures

Figure 4C. Please number the hierarchical clusters and genomic locations in this figure. They are referenced in the text.

Following your suggestion, we have labeled the hierarchical clusters and genomic locations in Figure 4C in the revised manuscript.

In addition, we revised the corresponding sentence in the main text as follows: "Biclustering reveals that modifications at g6 are specific to cluster C4, g7 to cluster C5, and g8 to cluster C6, while the first five genomic locations (g1 to g5) show similar modification patterns across all reads."

<https://doi.org/10.7554/eLife.104618.2.sa0>